

Human Computer Interaction

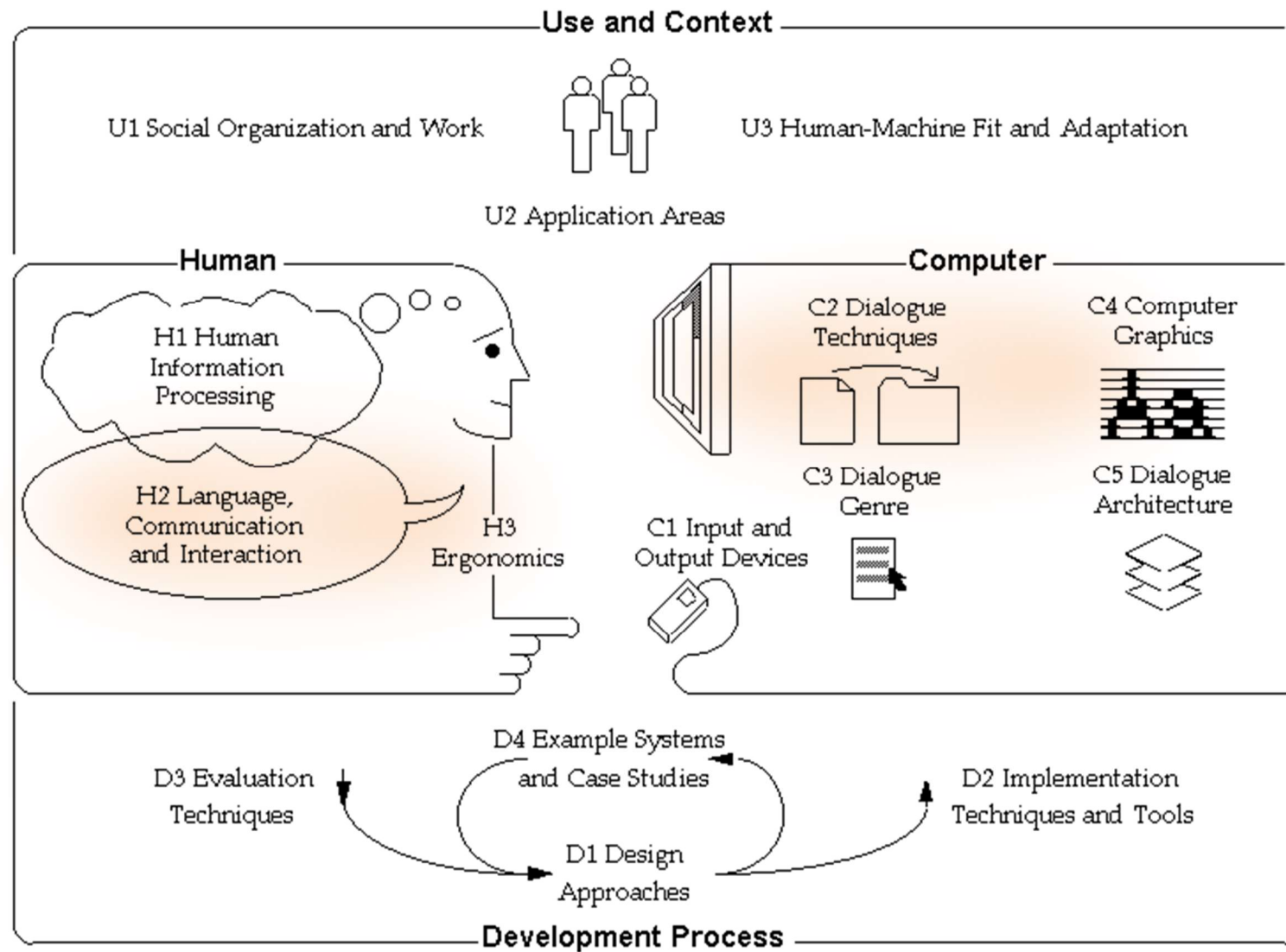
Johan F. HOORN and Jeff TANG

Lecture 04: Language, communication, and dialogue techniques

Part 1

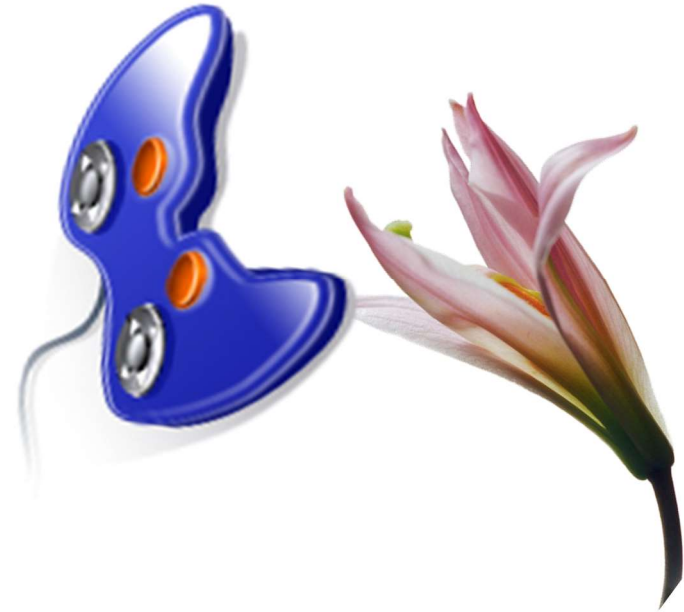


Overview



Contents

- ❑ Lecture's objectives
- ❑ Coding language
- ❑ Dialog languages
 - Dialog design
 - Graphical User Interface
- ❑ NLP: Natural Language Processing
- ❑ Human language



Slides in this lecture are partially based on those from Ms. Tiffany Tang, Dr. Vincent Ng, Dr. Grace Ngai, Dr. Saul Greenburg, and Alan Dix et al.

What you'll learn today

- ❑ Differences between types of languages (coding, dialog, NLP)
- ❑ How to talk to machines
- ❑ How to let machines talk to humans
- ❑ How machines and humans talk together
- ❑ How humans interpret machine talk



Talking to machines

□ Coding language

The fundamental difference between a computer and an ordinary machine is that a computer can be approached through language

Programming languages come in a variety of levels:

- Low-level: machine code, Assembly, etc.
- High-level: Java, Python, Ruby, etc.
- Compilers translate language X to language Y (e.g., objective-c to Swift)
- Program to another program (original definition: high-level to machine language)
- Specification to program
- Graph into program (i.e., visual programming)
- Program/specification to hardware

Talking to machines

❑ Coding language

- 1953: IBM develops the 701: all programming done in assembly languages
- Software costs exceed that of hardware!
- John Backus develops Speedcoding: an interpreted language that ran 20 times slower than hand-written assembly
- Backus' goal: translate high-level language to assembly language
- Many people thought this was impossible: Backus had failed in other projects before
- 1954-7 marks the FORTRAN I project
- By 1958: >50% of all software is in FORTRAN. Development time cut from weeks to hours
- A total number of 19 machines were built
- Software costs exceeded that of hardware!

Processor
(vacuum tubes)



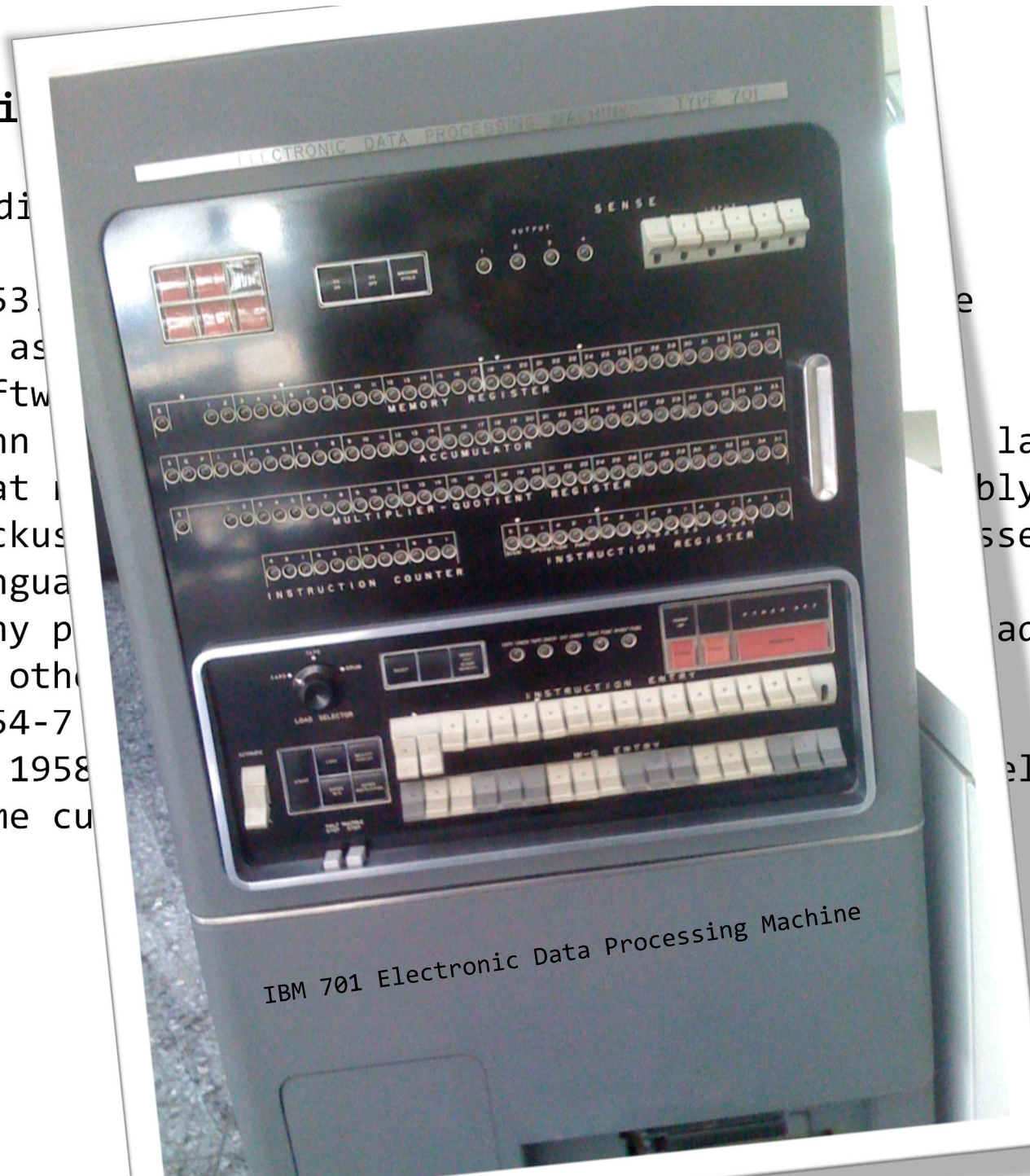
Operator's
console

Source: IBM

Talki

❑ Coding

- 1953
- in as
- Softw
- John
- that
- Backus
- langua
- Many p
- in oth
- 1954-7
- By 1958
- time cu



IBM 701 Electronic Data Processing Machine

e

language
bly
assembly

ad failed

elopment

Sample Code in FORTRAN

You never learned FORTRAN
but you can comprehend it!

```
program addNumbers
```

```
! This simple program adds two numbers  
implicit none
```

```
! Type declarations  
real :: a, b, result
```

```
! Executable statements  
a = 12.0  
b = 15.0  
result = a + b
```

```
print *, 'The total is ', result
```

```
end program addNumbers
```

Speedcoding and later FORTRAN took the majority of software programming in the late 1950s, despite of being *slower*!!

Lesson learned:

You can focus on ease of use at the expense of system resources

9 legacy programming skills still in demand

https://www.cio.com/article/3243575/9-legacy-programming-skills-still-in-demand

UNITED STATES DIGITAL MAGAZINE EVENTS CIO THINK TANK IDG TECH(TALK) COMMUNITY RESOURCE LIBRARY INSIDER

Home > Careers > IT Skills

FEATURE

9 legacy programming skills still in demand

Tech may change quickly, but companies still need to uphold legacy systems and networks. Here are the top nine legacy programming languages companies still hire for and support.

DB2, C, Cobol, Assembly, Perl, Delphi and Object Pascal, Fortran, REXX, Pascal

By Sarah K. White
Senior Writer, CIO | DEC 20, 2017 3:00 AM PST

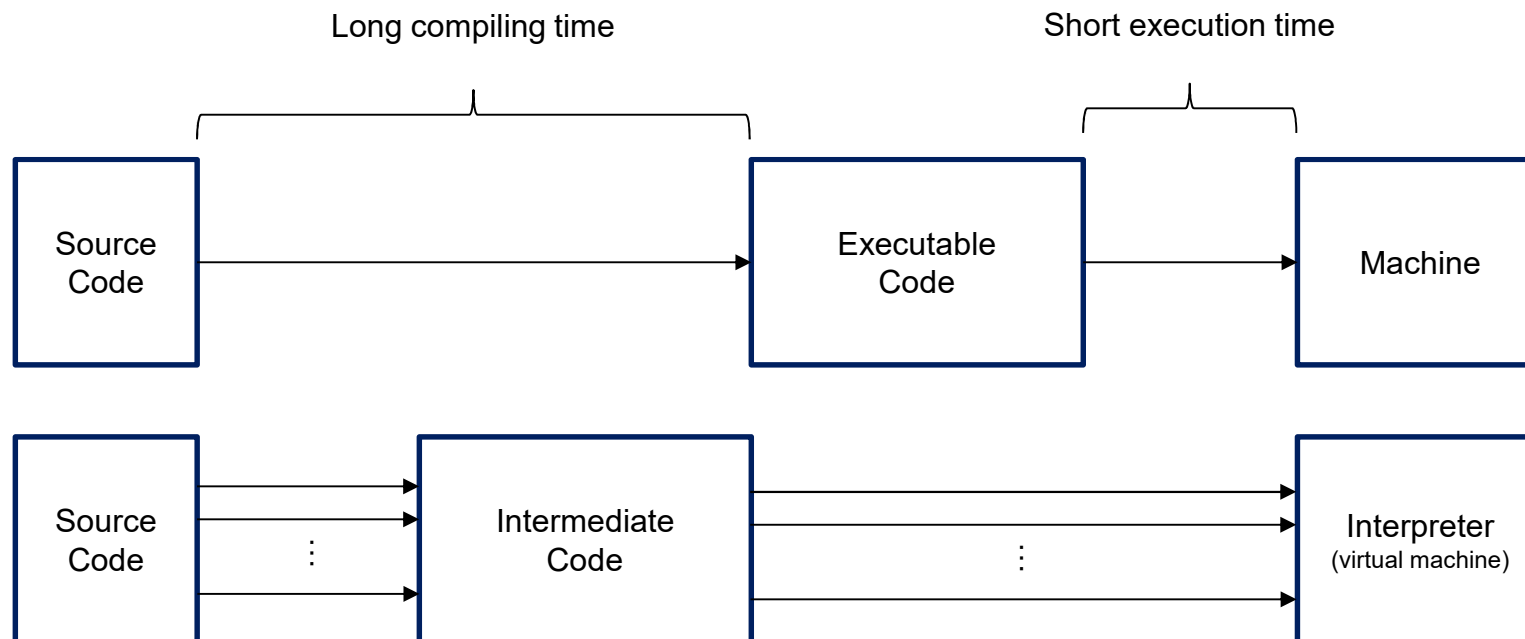


Typ hier om te zoeken

14:19
15-3-2021

Interpretation vs. Ahead of Time Compilation

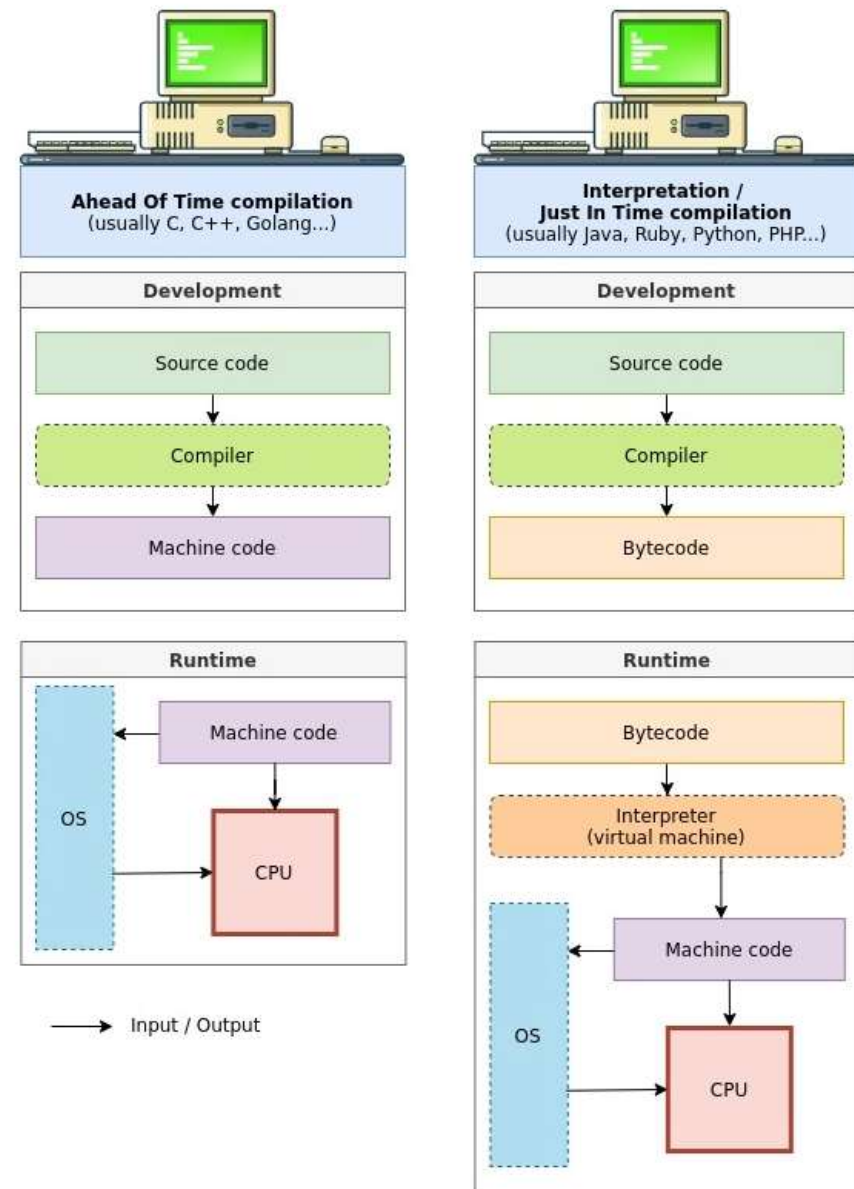
- (Conventional) interpreter translates just one statement of the program at a time
- Ahead of time compilation translates the entire program into machine code at once
- Many modern programming languages blend the two approaches (e.g., Java)



Interpretation vs. Ahead of Time Compilation

Pros and cons of interpretation (just in time compilation) and (ahead of time) compilation

- Cross-platform
- Compiling time
- Execution efficiency



Talking to machines

- Compiler architecture

- Two phases to compilation:

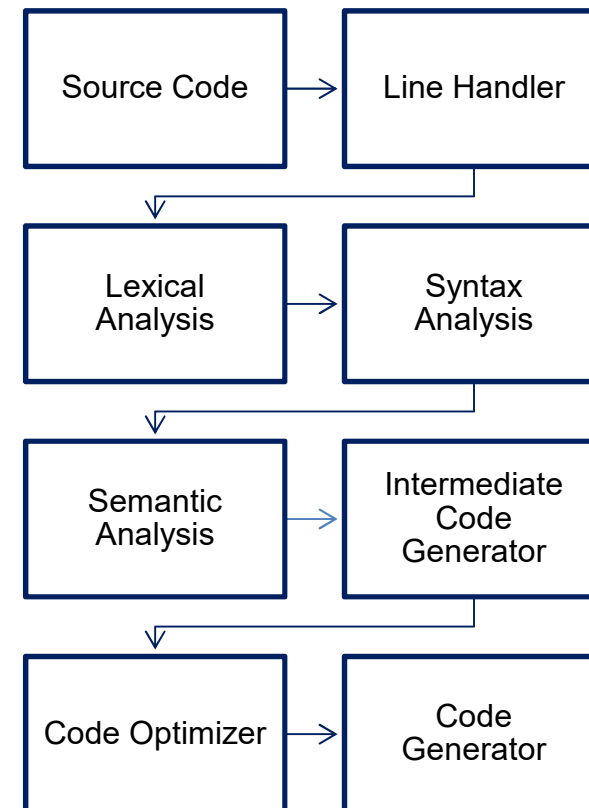
1. Analysis phase: Breaks source program into pieces to build up intermediate representation

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis

2. Synthesis phase: Generates target program from intermediate representation

- Intermediate Code Generator
- Code Optimizer
- Code Generator

- Data flow through a compiler



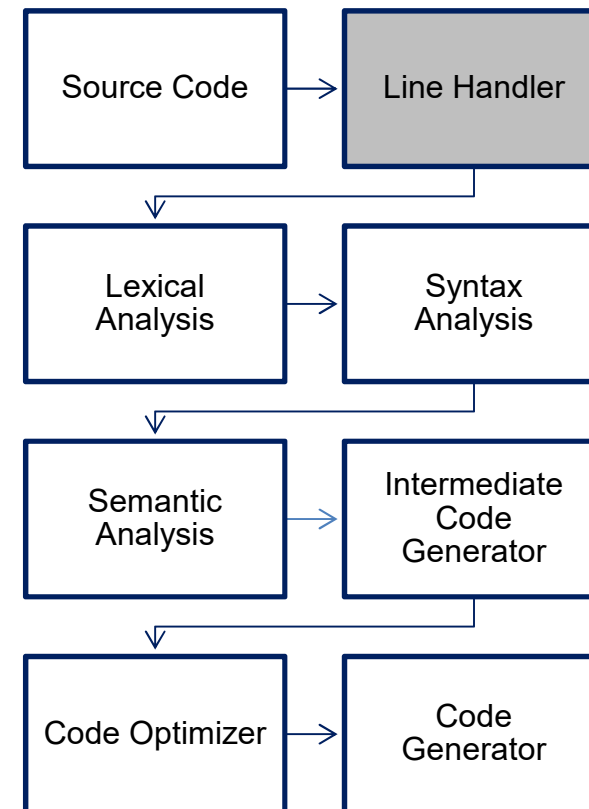
Talking to machines

Compilation: 1. Analysis phase

- Data flow through a compiler

Line handler

- Low-level routines that handle input/output from the source file
- Input line treated as array of characters, uses pointer to next character
- Skips over whitespace, ignores comments, handles continuation lines
- APIs used:
 - `getchar()`: gets next character from input line and move the pointer
 - `peekchar()`: gets next character from input line without moving pointer
 - `peek2char()`: gets second character from input line without moving pointer



Talking to machines

Compilation: 1. Analysis phase

- Data flow through a compiler

Lexical Analyzer

- The source file is simply a unique sequence of characters
- Lexical analyzer scans output from line handler from left to right
- Converts sequences of characters into “words”, or tokens

- Token: a categorized sequence of characters
- Example token types: identifier, reserved word, number, operator
- Example token values:

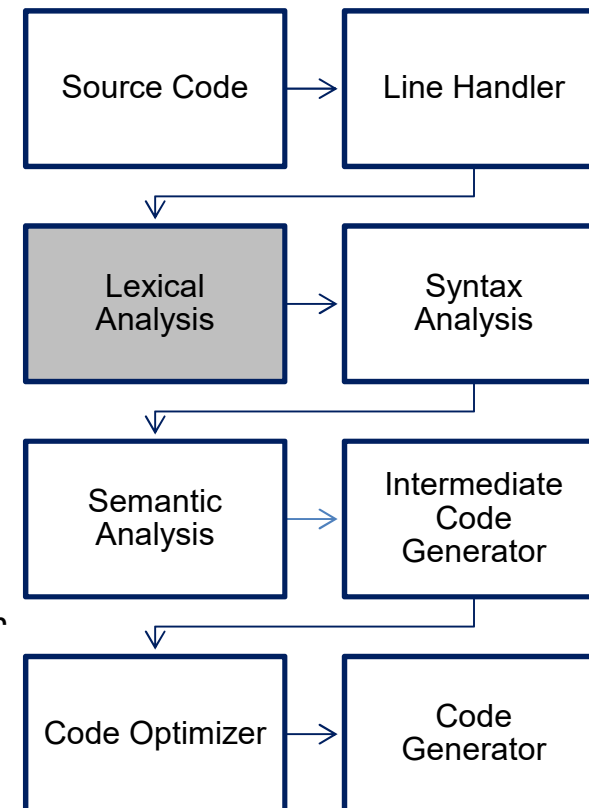
Identifiers: string and symbol table pointer

Reserved words: integer code

Numbers: binary form

Operators: integer code

- Sequence of characters that make up a token is called a *lexeme*



Talking to machines

□ Coding language

Compilation: 1. Analysis phase

Lexical Analysis Example

Program code: `position = initial + rate * 60;`

Result from lexical analyzer:

Lexeme	Token
position	Identifier
=	Operator
initial	Identifier
+	Operator
rate	Identifier
*	Operator
60	Number
;	Semicolon

Symbol / Token table

- Data structure used by compiler to associate tokens with their attributes
- Attributes include:
 - Type of the identifier
 - Storage allocated for the identifier
 - Scope of the identifier
 - Number and types of arguments (for function identifiers)
- Often implemented using a hash table

Talking to machines

Syntax Analyzer

- Syntactic Analysis is also called *parsing*
- Analyzes groups of related tokens that form larger constructs such as statements and expressions
- Converts linear string of tokens into *hierarchical structured* representation called a *parse tree*
- Checks for syntactic errors at this point
- Hierarchical structure expressed by recursive rules called *productions*

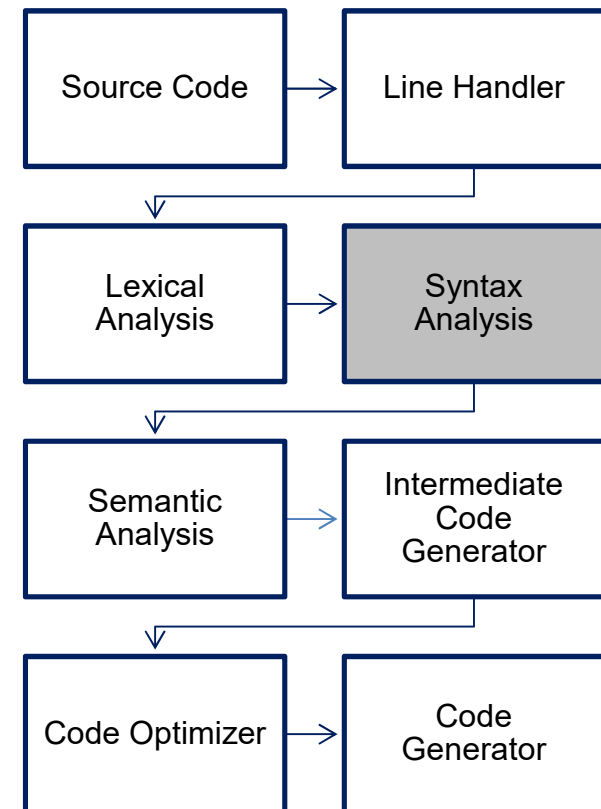
- Examples:

`<assignment> → identifier "=" <expr>`

`<expr> → identifier | number | <expr>`

`<operator> <expr>`

`<operator> → + | - | * | /`

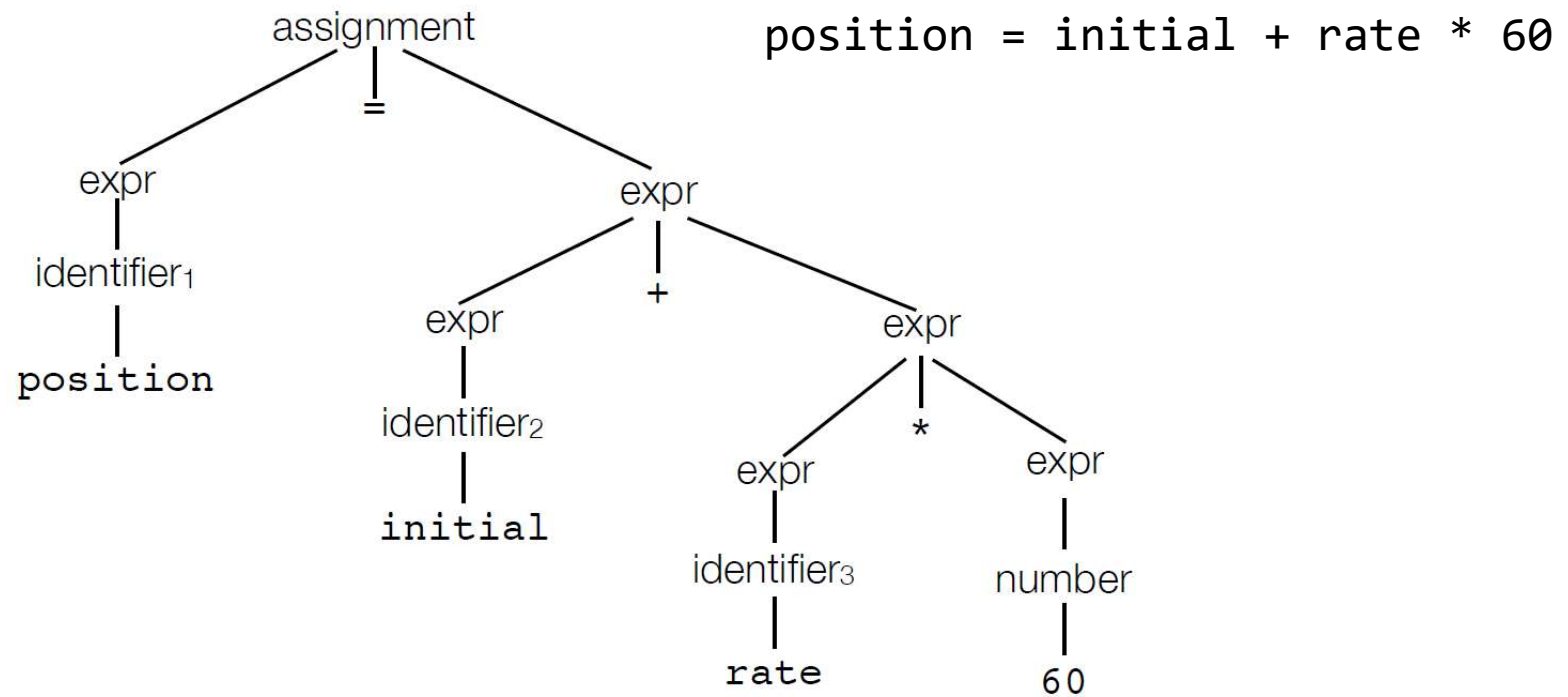


Talking to machines

□ Coding language

Compilation: 1. Analysis phase

Syntactic Analysis Example: a parse tree

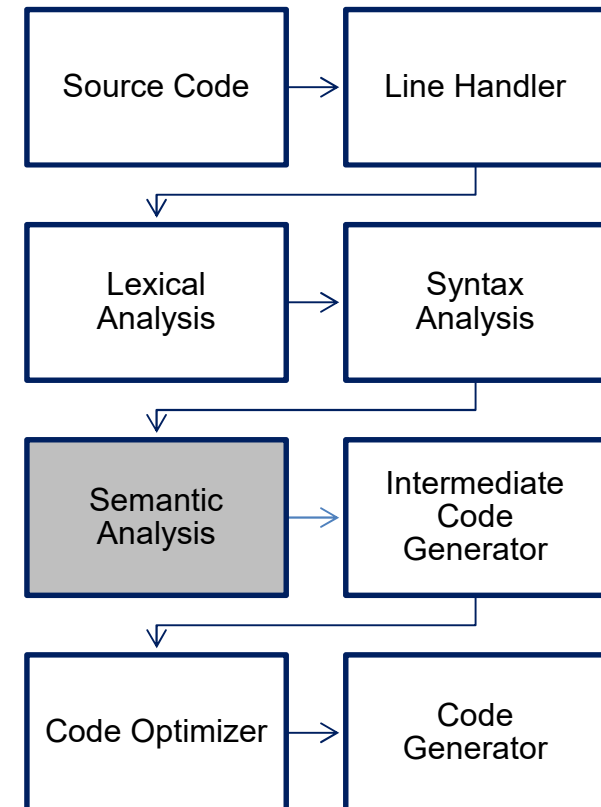


Semantic Analysis

- Concerned with the *meaning* of the program
- Checks for semantic errors, such as type errors
- Type checking:
 - Checks types of operands
(e.g., no real number as array index,
% operator requires integer arguments)
 - Insertion of extra operations such as
type coercion



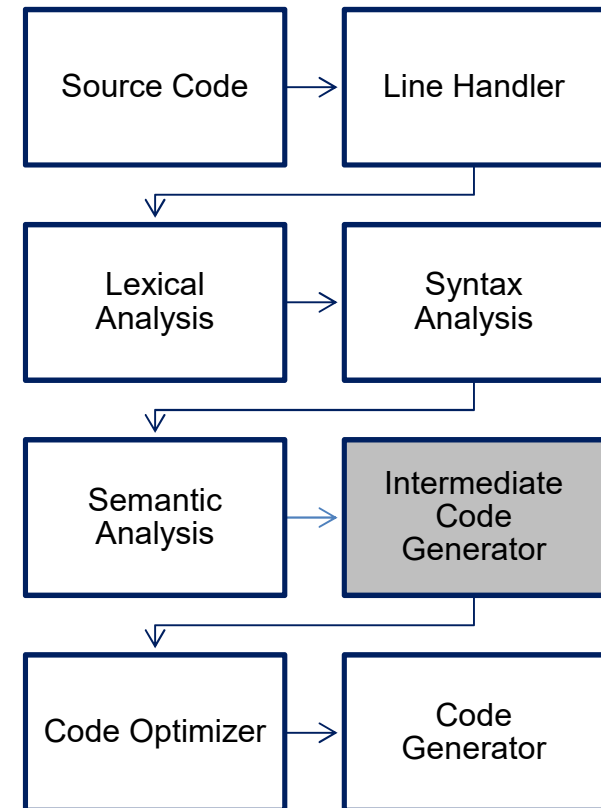
In view of the surrounding context, compiler implicitly and automatically changes a value of a certain type into a value of another type



Compilation: Synthesis phase

Intermediate Code Generation

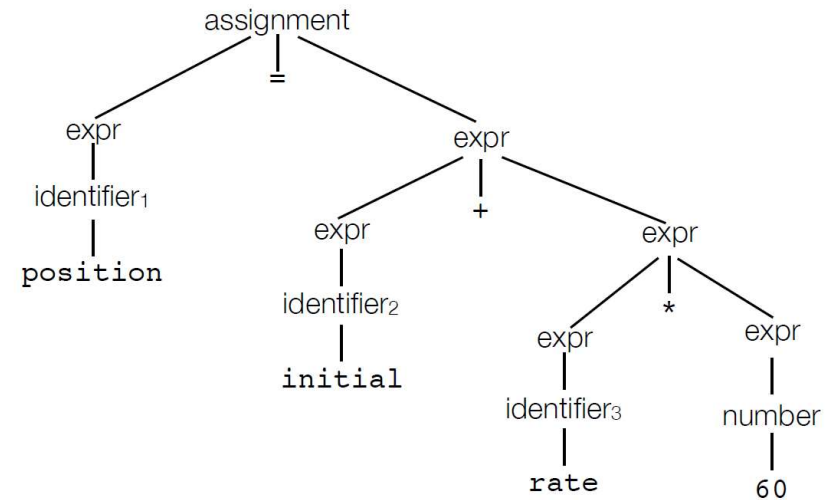
- Intermediate code is generated as a program for an “abstract machine”
- Needs to be easy to translate into the target program
- Uses *three-address code*, similar to Assembly:
 - Each instruction can be described as a 4-tuple: (operator, operand1, operand2, result)
 - And a general form:
result = operand1 op operand2
- Makes the operator precedence explicit
- Some statements have to be broken up into multiple statements; temporary names must be generated to cover intermediate results



Compilation: Synthesis phase

Intermediate Code Examples

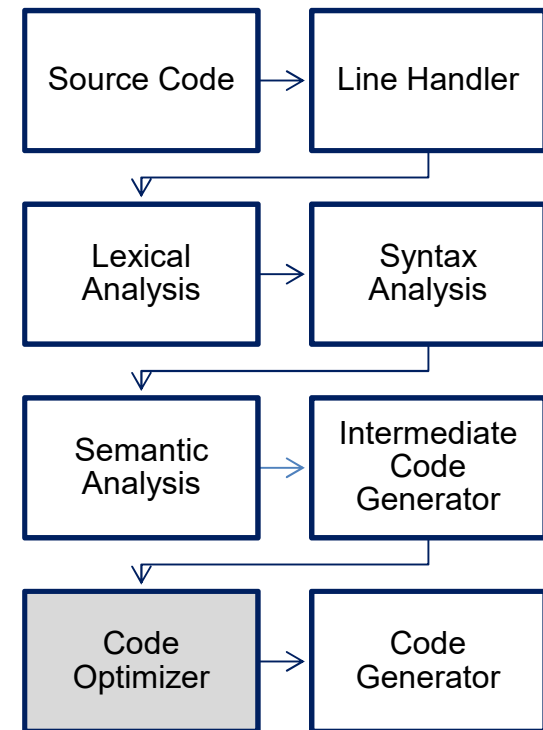
- Original Statement:
`position = initial + rate * 60;`
- Changed into:
`temp1 = float(60);`
`temp2 = identifier3 * temp1;`
`temp3 = identifier2 + temp2;`
`identifier1 = temp3;`



Code Optimization

- Improves intermediate code so that final machine code runs faster; or, can also optimize for program space (memory space) rather than run-time speed
- Different compilers do this differently; some are more efficient than others
- Some easy tricks:
 - Constant folding – perform operations on constants at compile time
 $2 * 3.14 \rightarrow 6.28$
 - Reduction in strength: $x^2 \rightarrow x * x$
 - Loop unrolling:

```
for (i = 1; i <= 3; i++)  
{ x[i] = y[i]; }  $\rightarrow$  x[1] = y[1]; x[2] = y[2]; x[3] = y[3];
```
- Obviously, optimized code should still produce correct results!



Code Optimization Example

```
position = initial + rate * 60;
```

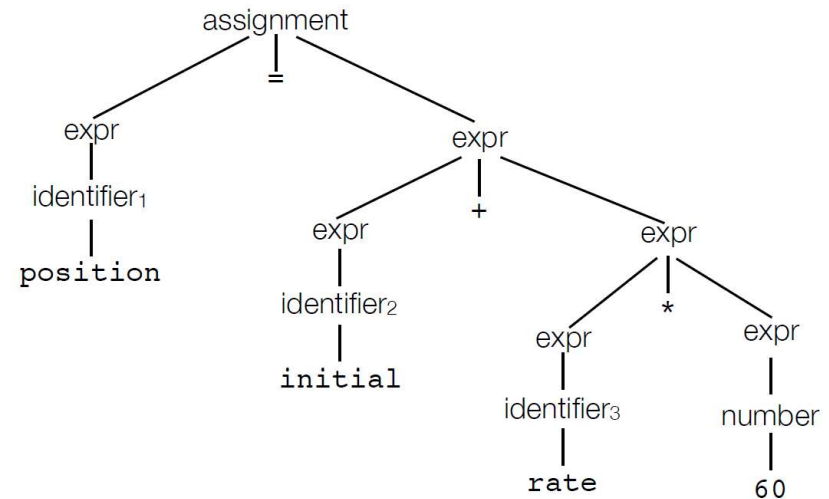
```
temp1 = float(60);  
temp2 = identifier3 * temp1;  
temp3 = identifier2 + temp2;  
identifier1 = temp3;
```



Optimization

```
temp1 = identifier3 * 60.0;  
identifier1 = identifier2 + temp1;
```

```
temp1 = rate times 60  
position = initial + rate times 60
```



```
temp1 = float(60);  
  
temp2 = identifier3 * temp1;  
  
temp3 = identifier2 + temp2;  
  
identifier1 = temp3;
```

Optimization

```
temp1 = identifier3 * 60.0;  
  
identifier1 = identifier2 + temp1;
```

Code Generation

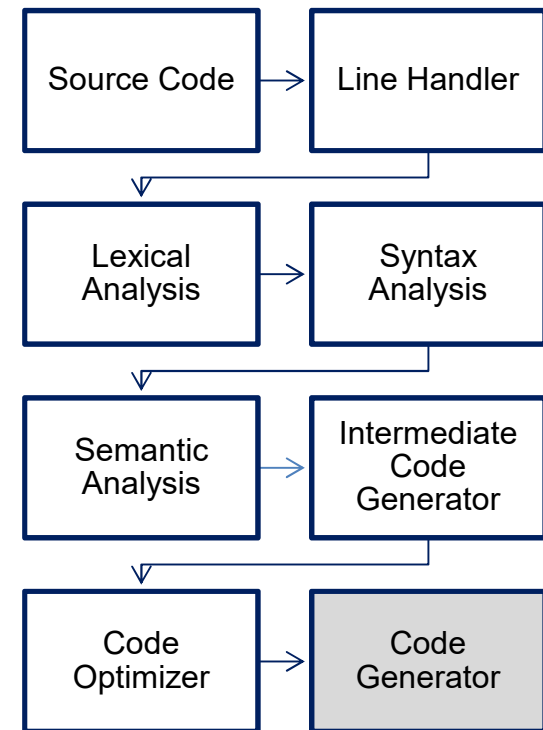
- Generates target code consisting of Assembly code

- Does the following:

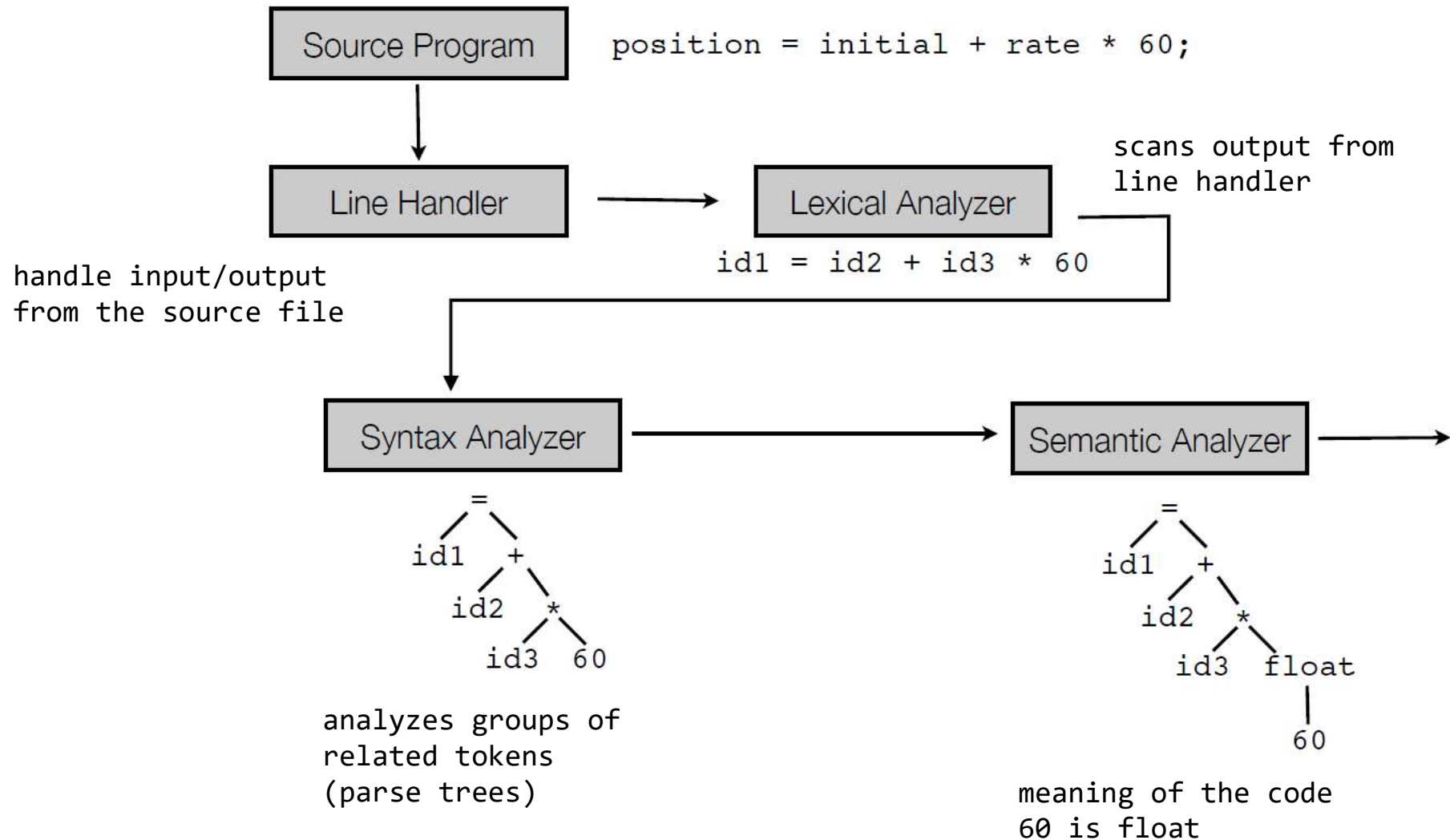
- Selects memory locations for each variable
- Translates instructions into sequence of assembly instructions
- Assigns variables and intermediate results to memory registers

- Examples:

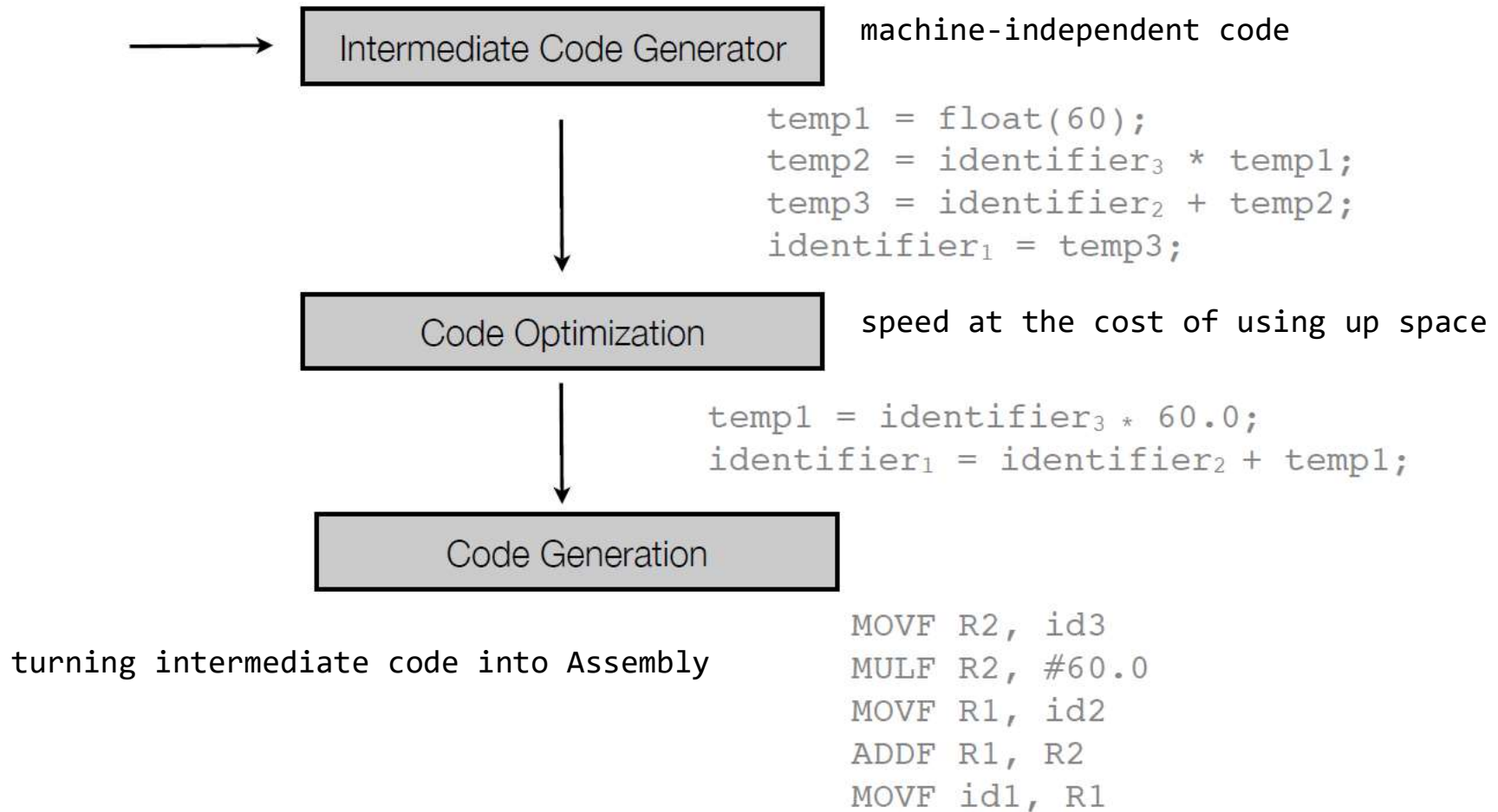
```
MOVF R2, id3      # F stands for floating-point instruction
MULF R2, #60.0    # “#” means that 60.0 is a constant
MOVF R1, id2      # First and second operand of each instruction
ADDF R1, R2       # Specify source and destination
MOVF id1, R1
```



Summary: Talking to machines



Summary: Talking to machines



5 minutes break

Connection interrupted

Please check your internet connection or hold a moment while we try to reconnect.

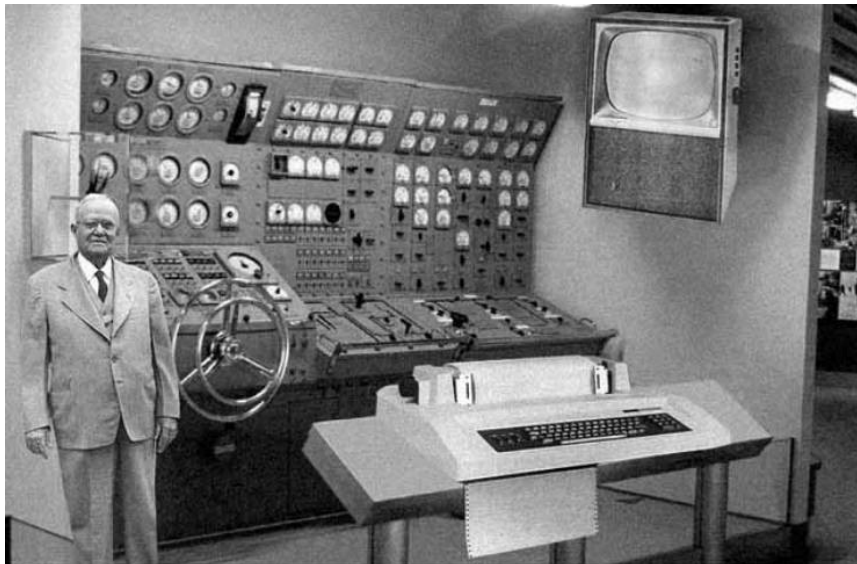
TRY NOW

**From talking to machines
to
Talking to humans**

Talking to machines and humans

□ Coding language

- Programming languages used to be the only interface that a computer had (think old-style punch cards and mainframes)
- Would programming languages make a good user interface today?



```
program addNumbers

    ! This simple program adds two numbers
    implicit none

    ! Type declarations
    real :: a, b, result

    ! Executable statements
    a = 12.0
    b = 15.0
    result = a + b

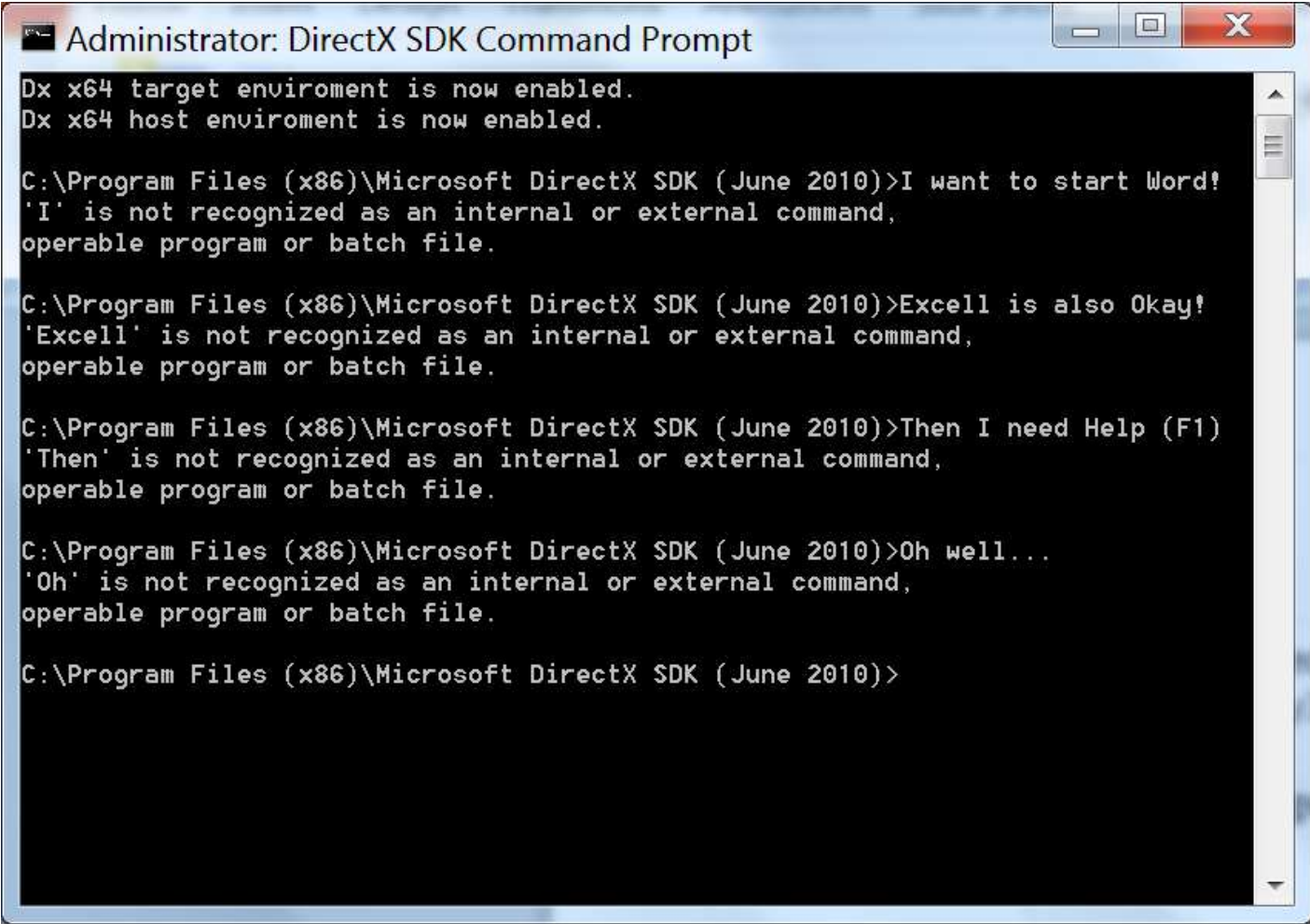
    print *, 'The total is ', result

end program addNumbers
```

*RAND: With the teletype interface and the Fortran language,
the computer will be easy to use*

Talking to machines and humans

- ❑ The prompt for a user interface?



```
Administrator: DirectX SDK Command Prompt
Dx x64 target enviroment is now enabled.
Dx x64 host enviroment is now enabled.

C:\Program Files (x86)\Microsoft DirectX SDK (June 2010)>I want to start Word!
'I' is not recognized as an internal or external command,
operable program or batch file.

C:\Program Files (x86)\Microsoft DirectX SDK (June 2010)>Excell is also Okay!
'Excell' is not recognized as an internal or external command,
operable program or batch file.

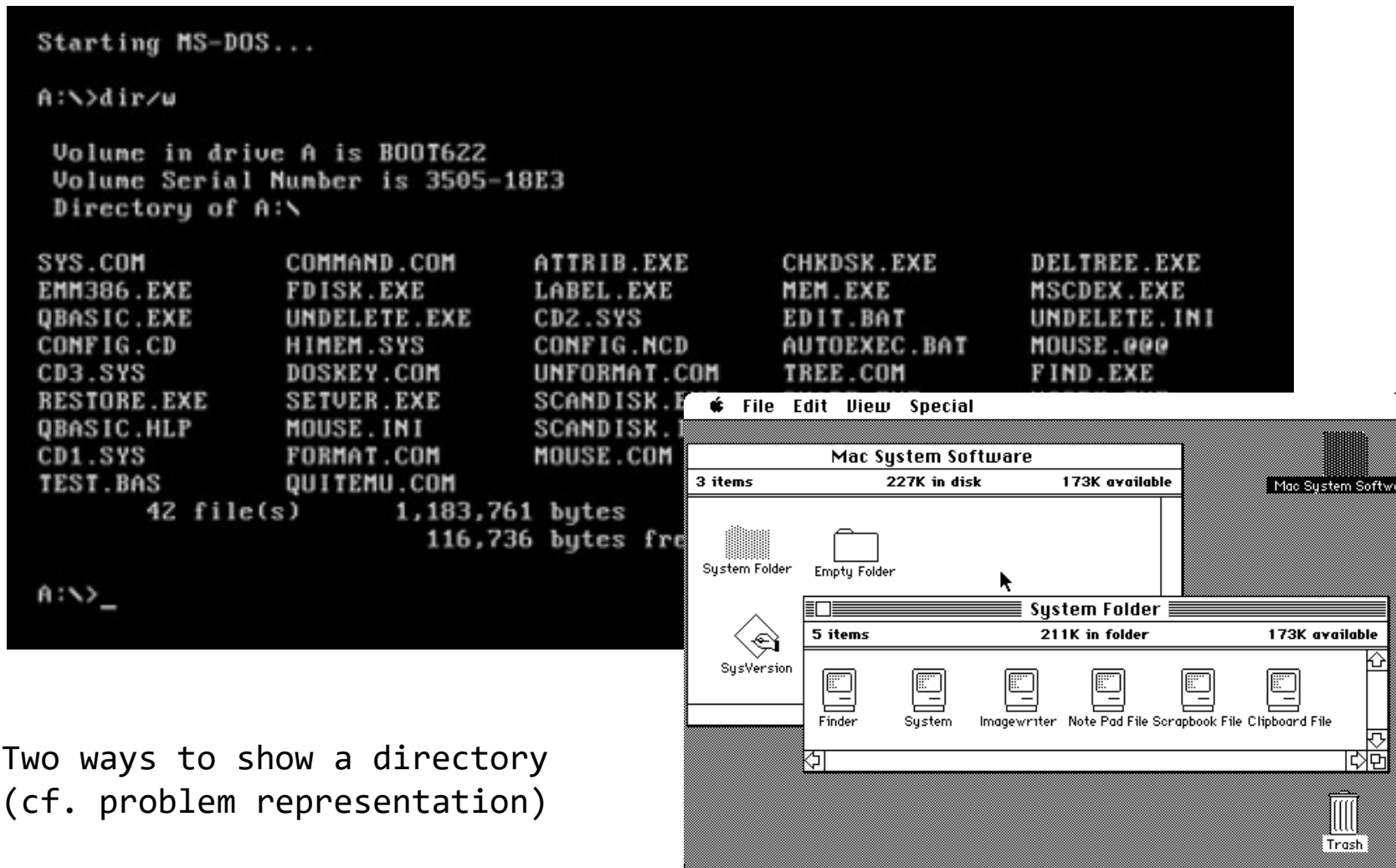
C:\Program Files (x86)\Microsoft DirectX SDK (June 2010)>Then I need Help (F1)
'Then' is not recognized as an internal or external command,
operable program or batch file.

C:\Program Files (x86)\Microsoft DirectX SDK (June 2010)>Oh well...
'Oh' is not recognized as an internal or external command,
operable program or batch file.

C:\Program Files (x86)\Microsoft DirectX SDK (June 2010)>
```

Talking to machines and humans

- ❑ The prompt for a user interface?



Two ways to show a directory
(cf. problem representation)

Talking to machines and humans

□ Dialog design

You will learn:

- Reasons for formal dialog specification
- Be able to use State Transition Networks (STNs) and Regular Expressions to specify a dialog
- Be able to conduct checks for completeness, reversibility, and dangerous states with dialogs
- Know about other methods for dialog specifications

Research Grants Council 研究資助局

General Research Fund and Early Career Scheme - Principal Investigator [jhoorn] ([Change Scheme / Role](#))

Dialog design?

[Logout](#)

PART V - ADDITIONAL ATTACHMENTS

Education Plan, Technology Transfer Plan, Letters of Collaboration

(Please select the appendix number and choose the file, then click **SAVE** to upload the file.)

(A maximum of 20 words for each box to caption each upload.)

- ☒ Appendix 1: [Education Plan](#) (up to one A4 page)
 - ☒ Appendix 2: [Technology Transfer Plan](#) (Optional)
 - ☐ Appendix 3: [Letters of Collaboration with other institutions](#)
- Caption of the document:

Word Count

- ☐ Appendix 4: [Letters of Collaboration with other institutions](#)
- Caption of the document:

```
https://cerg1.ugc.edu.hk/cergprod/static/edu_plan_grf.html - ...
cerg1.ugc.edu.hk/cergprod/static/edu_plan_grf.html

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-
8859-1">
<title>Education Plan</title>
</head>
<body>
<u><b>Education Plan</b></u><br>
Applicants should provide a proposal for activities that will
foster the integration of research and education. <br><br>
Please scroll down to the bottom of the webpage for uploading the
Education Plan.</p>
</body>
</html>
```

It is 2022!!

These windows stay with you if you don't explicitly click them away

```
https://cerg1.ugc.edu.hk/cergprod/static/technologyTransferPlans.html - Google C...
cerg1.ugc.edu.hk/cergprod/static/technologyTransferPlans.html

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:ui="http://java.sun.com/jsf/facelets">
<head>
<title>Technology Transfer Plan</title>
</head>
<body>
<u><b>Technology Transfer Plan (Optional)</b></u><br>
To take forward the Director of Audit's recommendation that a closer link with
the Innovation and Technology Fund (ITF) should be enhanced, applicants are
invited to submit an optional technology transfer plan at the application stage.
Any funded projects with technology transfer plans submitted will be passed to
the Innovation and Technology Commission, who will keep in view these projects
for possible ITF support in the future. The technology transfer plan will not
be assessed by the RGC for the purpose of funding allocation.
</p>
```

cerg1.ugc.edu.hk

Please select the appendix number and choose the file, then click **SAVE** to upload the file.

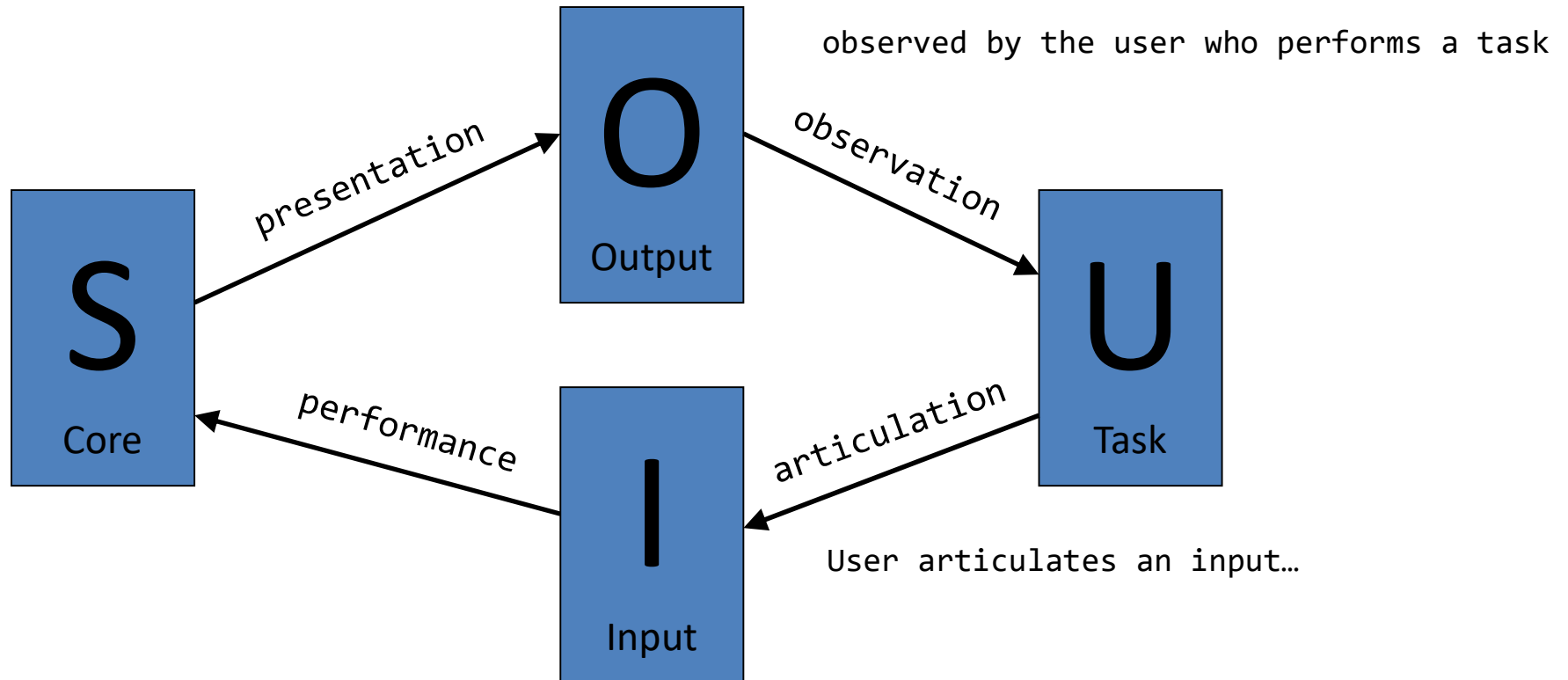
OK

Ignore this warning, which you get when you click **SAVE**

Talking to machines and humans

□ The goal of dialog design

System presents an output



... that makes the system perform

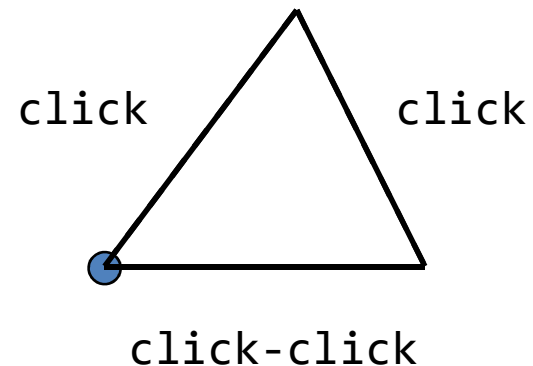
Talking to machines and humans

- The goal of dialog design

System presents an output observed by the user who performs a task

User articulates an input that makes the system perform

Example: Draw a polygon



Talking to machines and humans

□ Dialog design

A dialog is a conversation between two or more parties, usually cooperative

In user interfaces

- refers to the structure of the interaction
- syntactic level of human-computer 'conversation'

Levels:

- lexical – shape of icons, actual keys pressed, characters
- syntactic – order of inputs and outputs (keystrokes, clicks)
- semantic – effect on internal application/data, meaning of keystrokes, clicks

Talking to machines and humans

□ Dialog design. Why?

In a big system, analyze the dialog

- Can the user always get to see current shopping basket?

When you change platforms (e.g., Windows/Mac)

Dialog notations help us to

- Analyze systems
- Separate lexical from semantic aspects

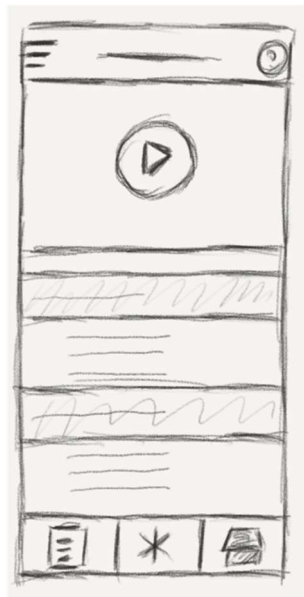
Before the system is built

- Dialog notations help us understand proposed designs (i.e. rapid prototyping)

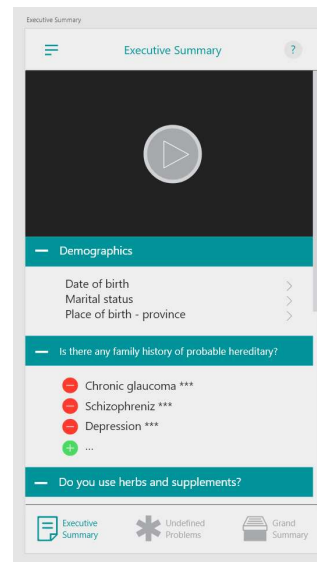
To discuss your system with clients / stakeholders

- Turn ideas into code
- Systematic testing
- 1000% productivity gain
- Formalism saves time!!

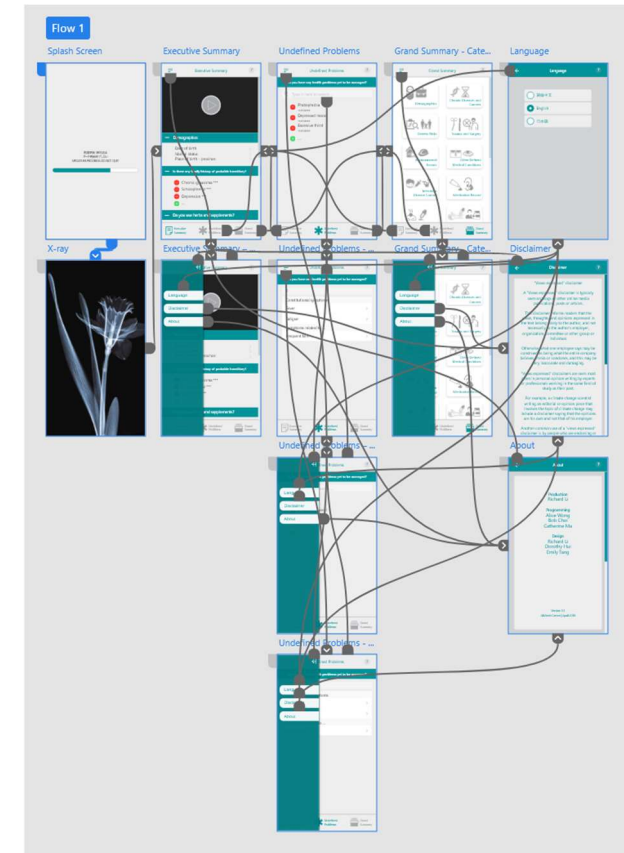
Dialogue Design for Prototyping



Sketch on
paper



Adobe XD



Talking to machines and humans

□ Dialog design techniques

Human-computer dialog very constrained

Some human-human dialog formal too (cf. legal, financial)

How do we describe dialogs ('notations')?

- State Transition Networks (STN)
 - State diagrams (UML)
 - Flow charts
 - Petri nets
 - JSD diagrams
- (Jackson Structured Design Diagrams)
- Diagrammatic
- Formal Grammars
 - Production Rules
 - CSP
- Textual

Linked to system semantics (what it does)
and system presentation (how it looks)

Talking to machines and humans

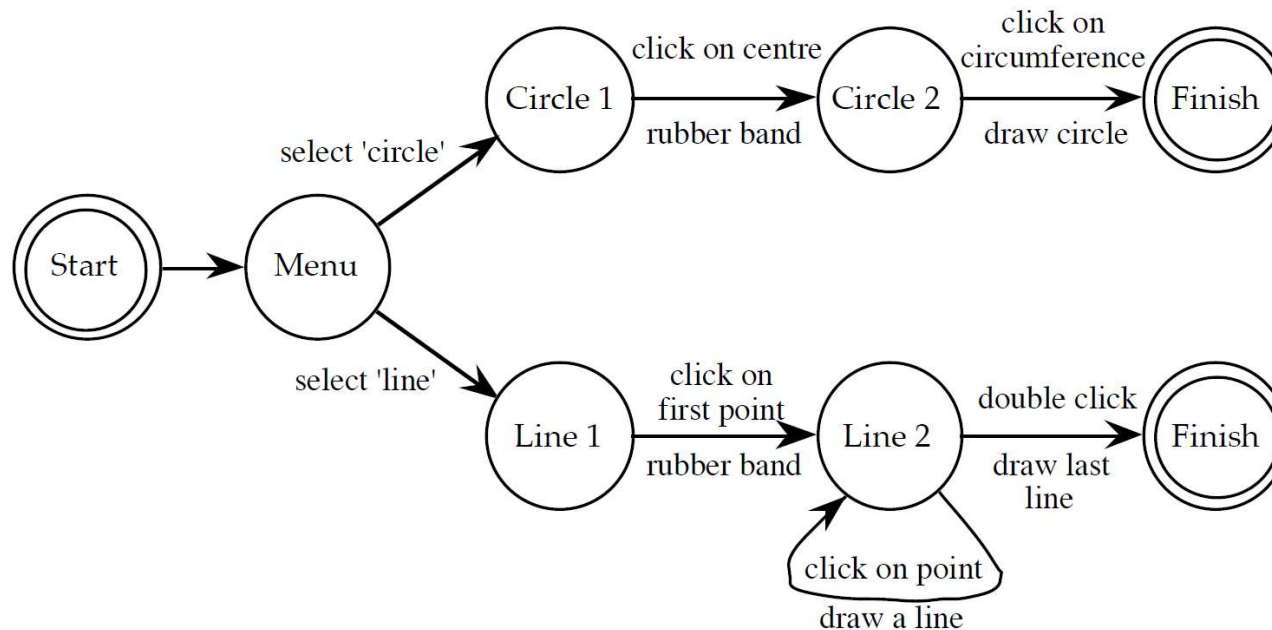
□ Dialog design: State Transition Networks (STNs)

Formal descriptions to analyze

- Inconsistency
- Reversibility
- Completeness
- Potential errors

State Transition Networks (STNs)

- Circles: States
- Arcs: Actions/Events

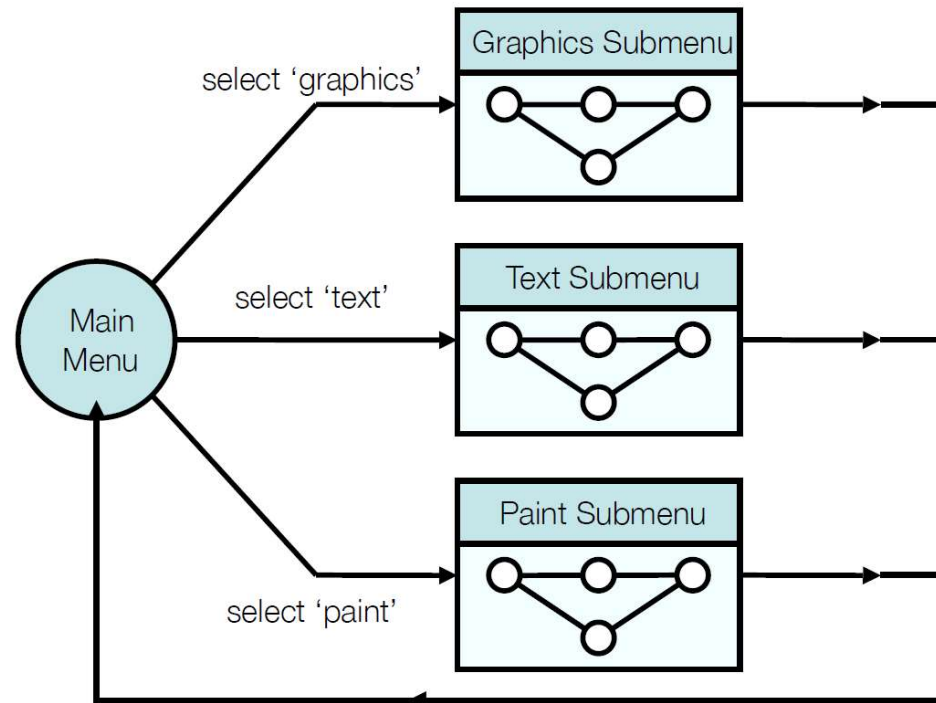


Talking to machines and humans

□ Dialog design: STN

Hierarchical STNs

- Really just an STN inside another STN
- Named sub-dialogs
- Essential for managing complex dialogs

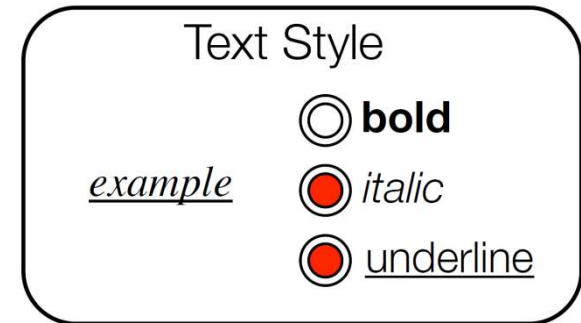


Talking to machines and humans

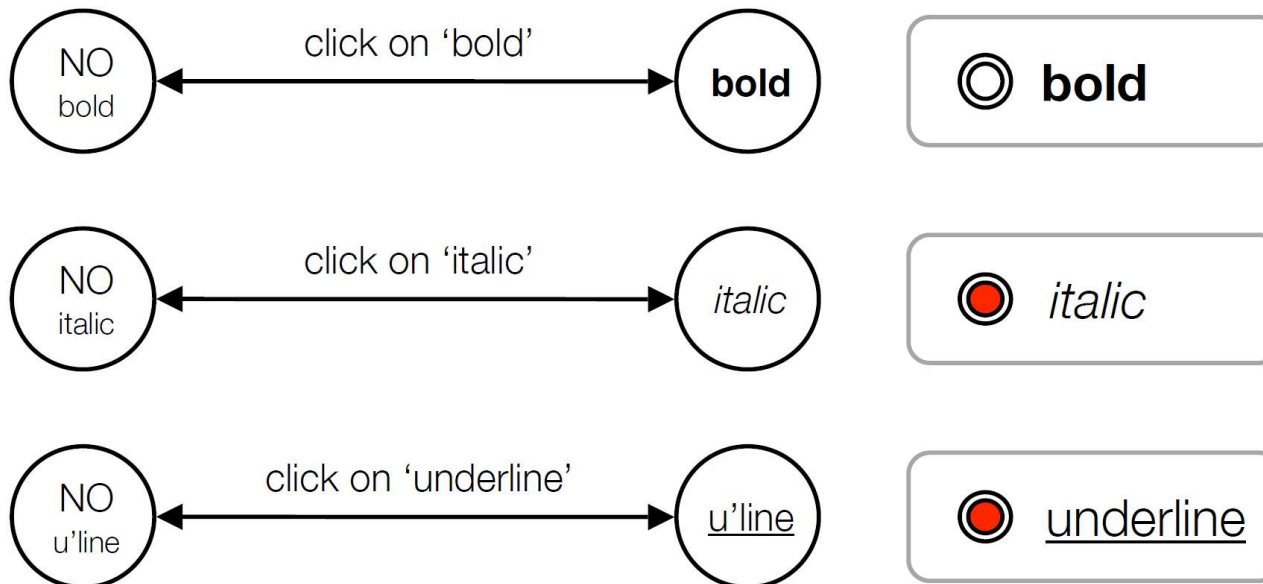
□ Dialog design: STN

Concurrent Dialogs

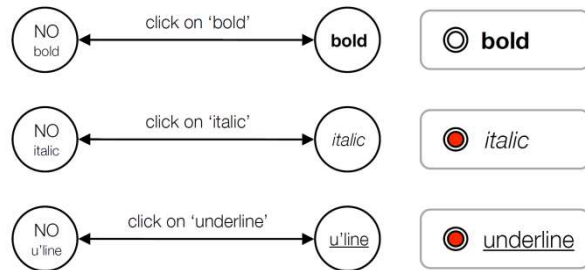
- Several things happen simultaneously
- Example: Simple dialog box for text formatting



Three toggles, three STNs:

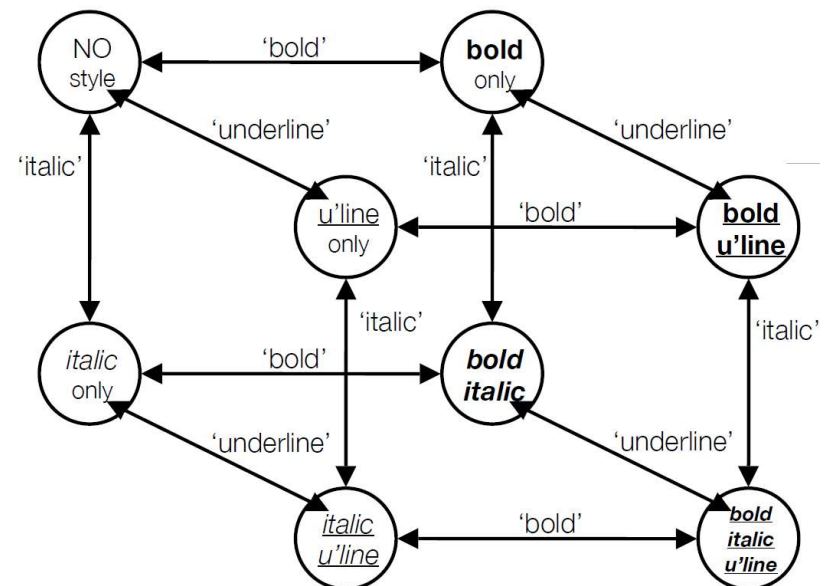
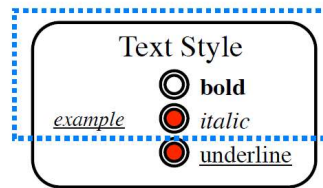
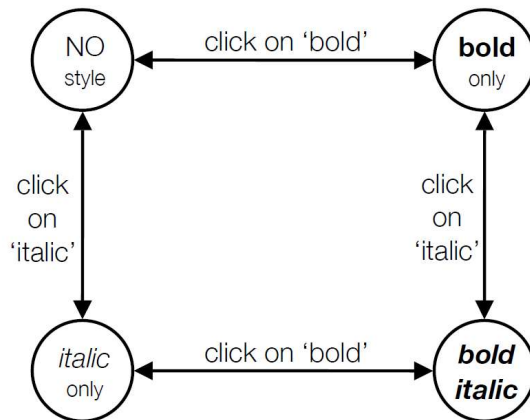


Talking to machines and humans



Concurrent Dialogs

- Put STNs together
- Beware of combinatorial explosion



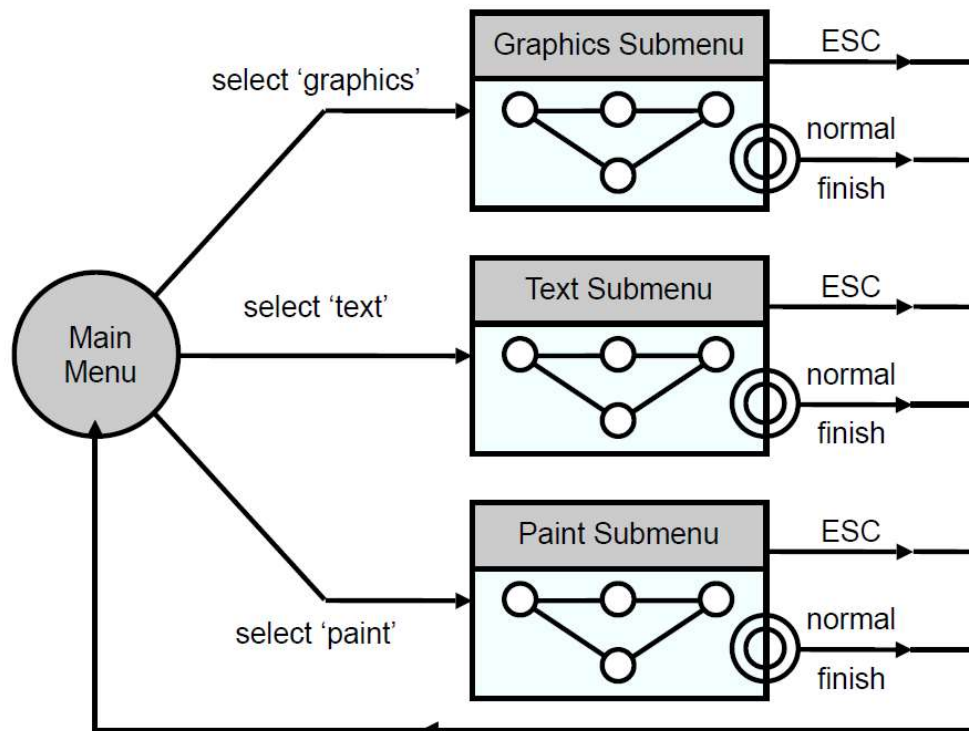
Imagine adding a fourth or fifth option:
 strikethrough: $2^4 = 16$ states
 shadow: $2^5 = 32$ states
 etc.

Talking to machines and humans

□ Dialog design: STN

Design of Escapes

- 'back' in Web, Escape/cancel keys
 - Similar behavior everywhere
 - End up with spaghetti of identical behaviors!



```
public class GameManager : MonoBehaviour
{
    private bool _gameOver = false;

    void Update()
    {
        if (Input.GetButton("Restart") && _gameOver == true)
        {
            SceneManager.LoadScene(1); // 1 is the game scene now
        }

        // NEW: if player presses exit button (escape or back), exit the game
        if (Input.GetButton("Cancel"))
        {
            Application.Quit();
        }
    }
}
```

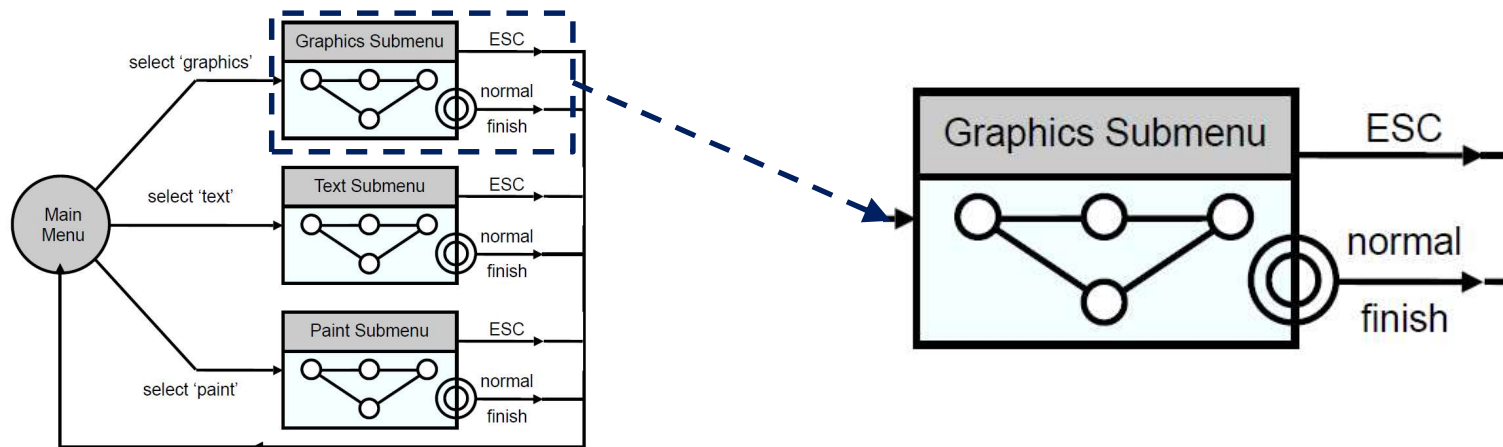
Talking to machines and humans

□ Dialog design: STN

Design of Escapes

- How do we show this?

Use hierarchical menus: 'normal' exit for each submenu plus separate escape arc active 'everywhere' in submenu

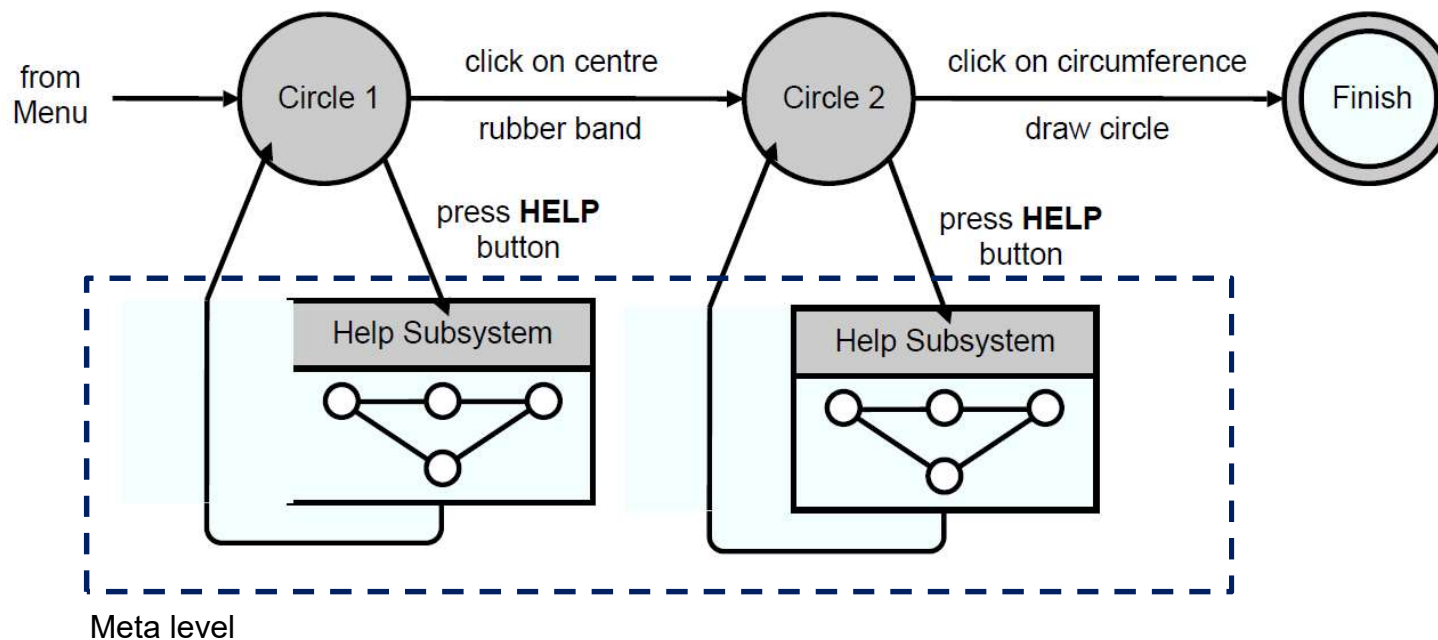


Talking to machines and humans

□ Dialog design: STN

Design of Help menus

- Menu structure nearly the same everywhere
- But return to same point in dialog could specify on STN but is very messy
- Usually best added at a 'meta' level

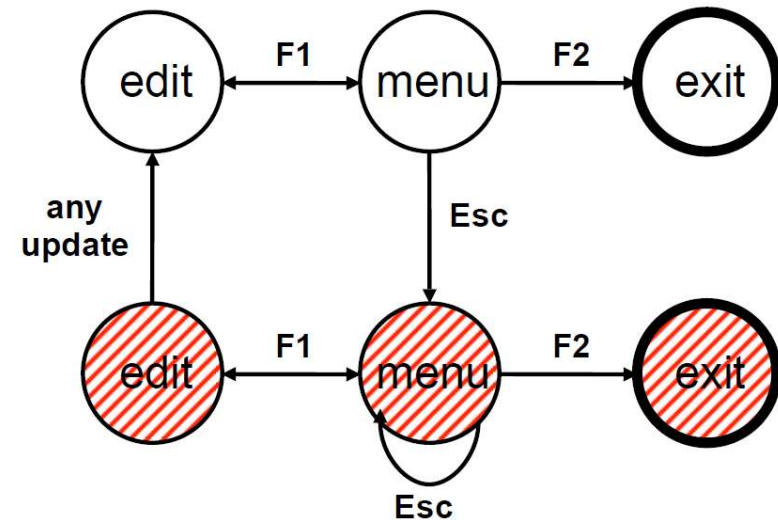
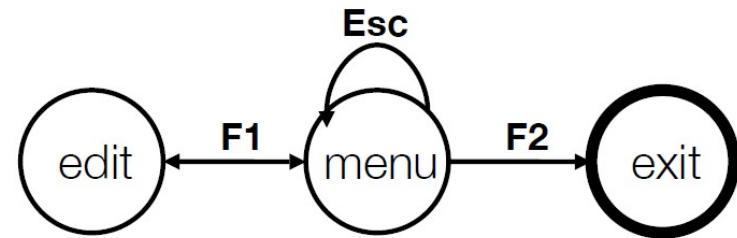


Talking to machines and humans

□ Dialog design: STN

Dangerous States

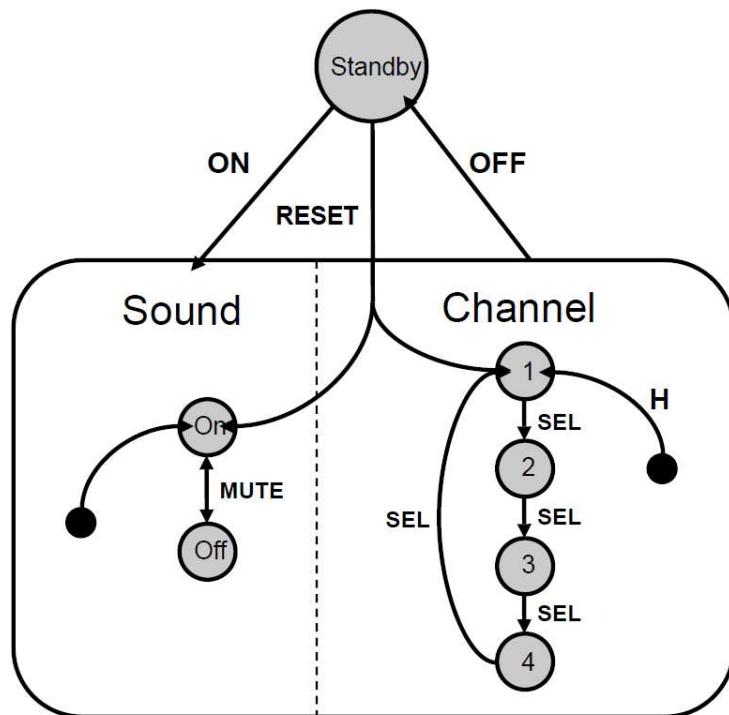
- Two-mode word processor:
 - F1: changes mode
 - F2: exit (and save)
 - Esc: no mode changes
 - But – Esc also resets **autosave**
-
- Exit with/without save cause dangerous states
 - Duplicate states with semantic distinction:
F1-F2 - exit with save
F1-Esc-F2 - exit with no save



Talking to machines and humans

□ Dialog design: State Diagram

- Used in UML
- Extension to STN (hierarchy, concurrent sub-nets, escapes)
- OFF always active (history states)
- Link marked H goes back to last state on re-entering subdialog



Visually specifies complex reactive systems

State transition network

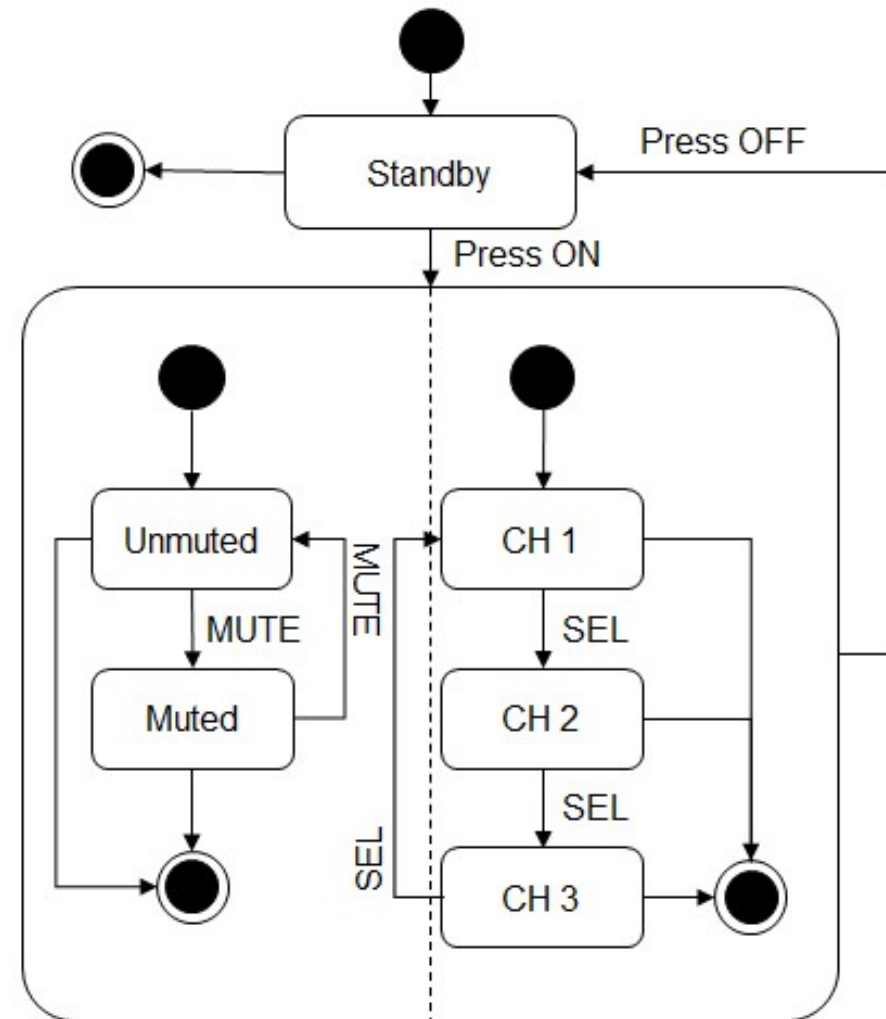
Hierarchy of STN's organized as charts

H designates states remembered

Talking to machines and humans

❑ Dialog design: State Diagram

- Used in UML
- Behavioral diagram
- Nested state machines (history states)
- Concurrent states



Solid circle - initial state
Concentric circles - final state
Arrowed lines - transition

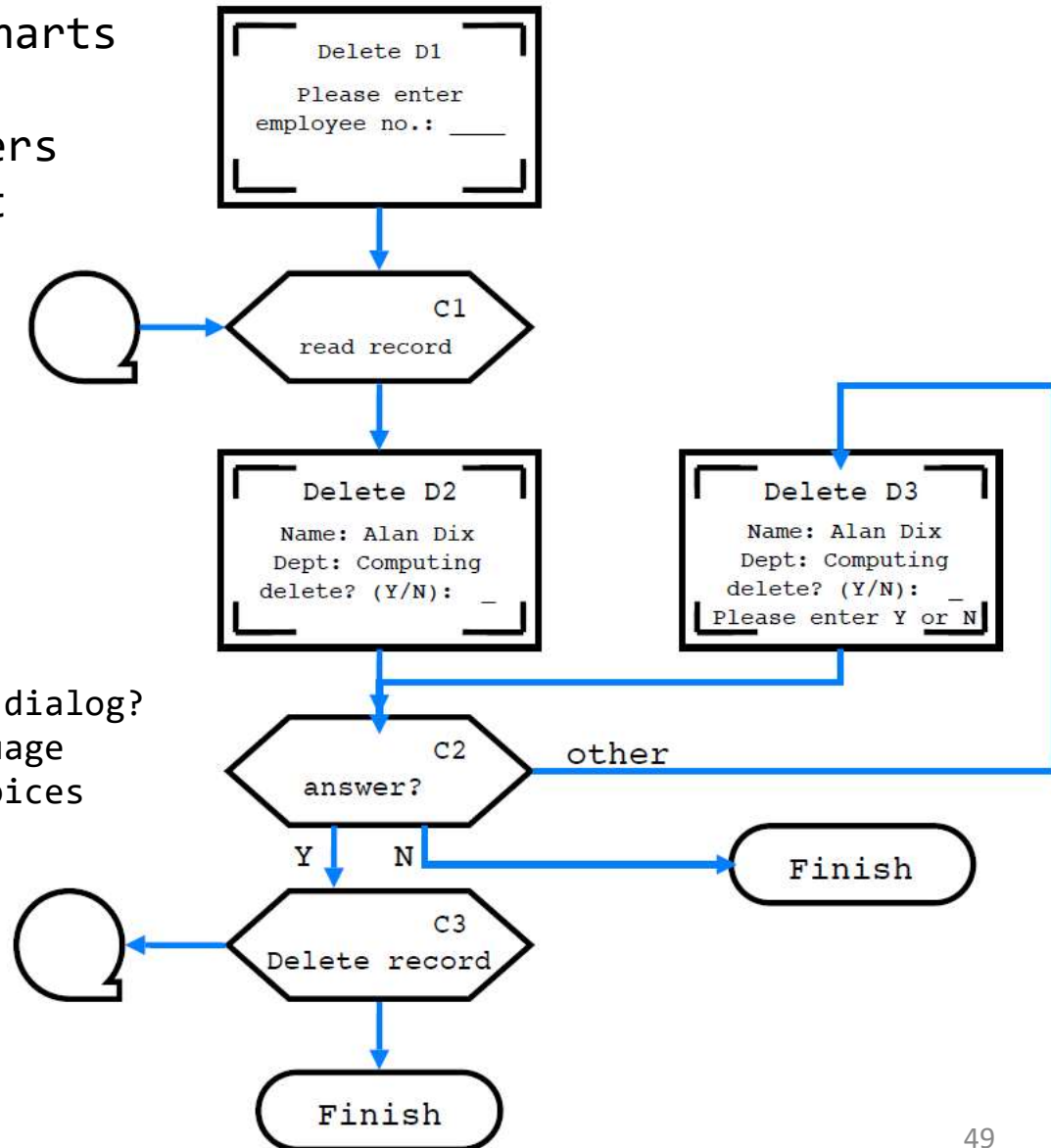
Talking to machines and humans

□ Dialog design: Flowcharts

- Familiar to programmers
- Boxes – Process/event (not state)

How can flow charts indicate dialog?

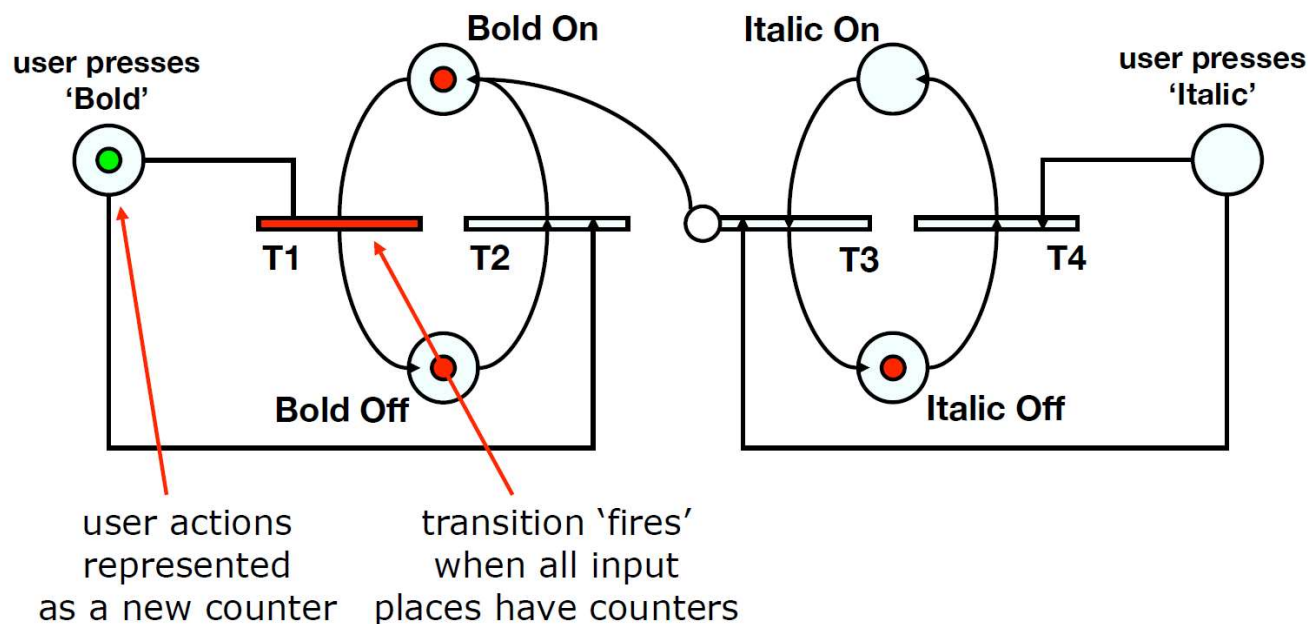
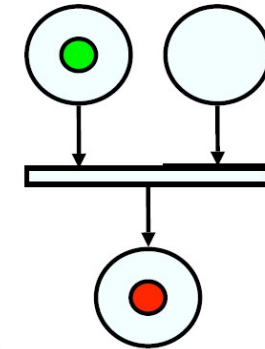
- Boxes indicate output language
- Diamonds indicate input choices
- Ellipses indicate states



Talking to machines and humans

❑ Dialog design: Petri Nets

- One of the oldest notations in computing
 - Places: comparable to 'states'
 - Transitions: switches to change states ('arcs')
 - Counters: place where user is in state diagram
 - Several counters are allowed for concurrent dialog states
 - Used for UI specification
- 



Talking to machines and humans

❑ Dialog design: Jackson Structured Design (JSD) Diagrams

- For tree-structured dialogs
- Less expressive
- Greater clarity

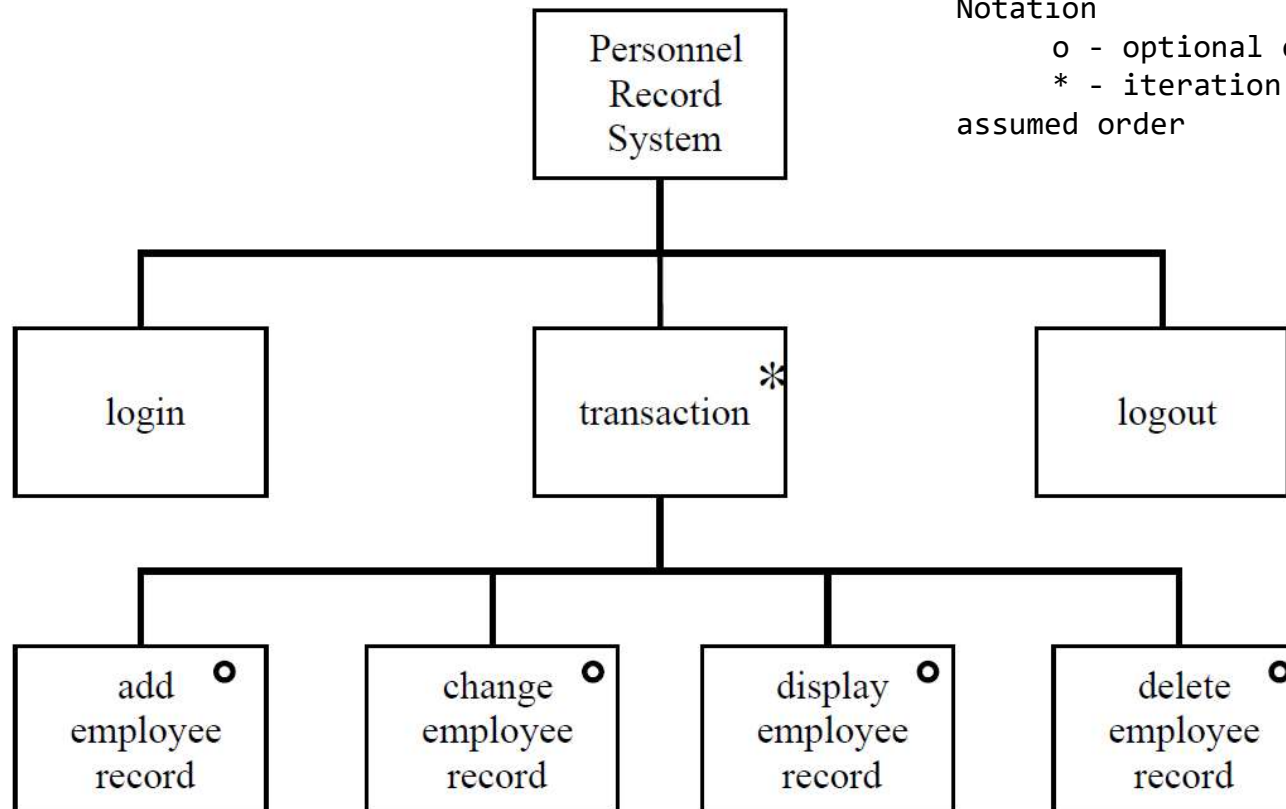
Hierarchical Task Analysis meets dialog design

Notation

o - optional elements

* - iteration

assumed order



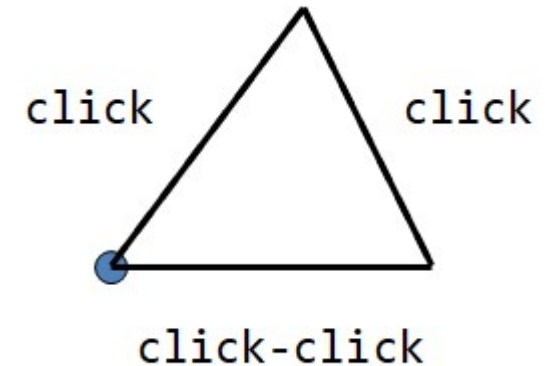
Talking to machines and humans

□ Dialog design: Textual – Grammars

- Regular expressions

`sel-line click click* dble-click`

- Same computational cost as STNs



Grammars: Backus–Naur form or Backus normal form (BNF)
metasyntax notation for context-free grammars

```
expr ::= empty
      | atom expr
      | '(' expr ')' sentence
```

- More powerful than regular expressions or STNs
- Still cannot handle concurrent dialogs

Talking to machines and humans

□ Dialog design: Textual – Grammars

Production rules

Unordered list of rules

if condition then action

- Condition based on state or pending events
- Every rule always potentially active
- Good for concurrency
- Bad for sequence

Sel-line → first

C-point first → rest

C-point rest → rest

D-point rest → <draw line>

- Events are added to list of pending events
- “first” and “rest” are internally generated events
- Bad at state!

Talking to machines and humans

□ Dialog design: Textual – Grammars

Propositional Production System

- State based
- Attributes:

Mouse: { mouse-off, select-line, click-point, double-click}

Line-state: { menu, first, rest }

- Rules:

```
select-line → mouse-off first
click-point first → mouse-off rest
click-point rest → mouse-off rest
double-click rest → mouse-off menu
```

- Bad at events!

Talking to machines and humans

□ Dialog design: Textual – Grammars

Communicating Sequential Processes (CSP) and Process Algebras are formal languages for describing patterns of interaction in concurrent systems

- Good for sequential dialogs

```
Bold-toggle = select-bold? → bold-on → select-bold?  
→ bold-off → bold-toggle  
Italic-toggle = ...  
Under-toggle = ...
```

- And concurrent dialog

```
Dialog-box = Bold-toggle || Italic-toggle || Under-  
toggle
```

- But causality unclear

Virtual Tutor Summary



Unlike an ordinary machine, a computer can be approached through language. However, coding language is a compromise between grammar-oriented assembly instructions and semantics-oriented human language

To explain a system to both coders and users, many dialog notations have been developed. They can be diagrams such as Petri Nets, flowcharts, and State Transition Networks. They can also be textual such as grammars and production rules

Selecting one or the other notation is a trade-off between focusing on events or states, of power vs clarity, model vs notation, of emphasizing sequential or concurrent states

Virtual Tutor Summary



For dialog design, you may want to use

- Diagrammatic approaches
- STN, JSD, Flowcharts
- Textual
- Grammars, production rules, CSP

Some of the issues you should think over (trade-offs you will face):

- Event-based vs. State-based
- Power vs. Clarity
- Model vs. Notation
- Sequential vs. Concurrent

